

Sistema Inteligente para la Identificación y Seguimiento de los derechos de los NNA por feminicidios

Trabajo Terminal II

Morales Martinez Hector Alberto

Directores:

M. en C. Ulises Saldaña Velez

M. en C. Gabriel Hurtado Avilés

Ciudad de México, a 4 de junio de 2026



0.1. Glosario y Conceptos Técnicos del Sistema	1
1. Introducción	3
1.1. Presentación del proyecto	3
1.2. Planteamiento del problema	4
1.2.1. La inviabilidad del monitoreo manual	4
1.3. Objetivos del Proyecto	4
1.3.1. Objetivo General	4
1.3.2. Objetivos Específicos	4
2. Especificación de requerimientos del sistema	5
2.1. Casos de uso del sistema	5
2.2. Requerimientos Funcionales del Motor Back-end	7
2.2.1. RF — Subsistema de Recolección de Datos	7
2.2.2. RF — Subsistema de Deduplicación en Cascada	8
2.2.3. RF — Subsistema de Análisis Semántico Neuro-Simbólico	9
2.2.4. RF — Subsistema de Agrupamiento Semántico (BERTopic)	10
2.2.5. RF — Subsistema de Persistencia Relacional	11
2.3. Requerimientos No Funcionales	12
3. Justificación y Alcance del Proyecto	13
3.1. Justificación Social	13
3.2. Justificación Tecnológica	14
3.3. Justificación ética y moral: Protección de la privacidad de los NNA	14
3.3.1. El interés superior del niño	15
3.3.2. Naturaleza de la información procesada	15
3.3.3. ¿Por qué el sistema no proporciona nombres de NNA?	15
3.3.4. Fortaleza ética del enfoque adoptado	16
3.3.5. Compromiso de privacidad	16
3.4. Alcance del Sistema	16
3.4.1. Alcance funcional	16
3.4.2. Alcance geográfico y temporal	17

3.5. Limitaciones y exclusiones	17
3.5.1. Exclusiones explícitas del alcance	17
3.5.2. Limitaciones técnicas	17
3.6. Alineación con el protocolo	18
4. Marco Teórico	19
4.1. Procesamiento de lenguaje natural y representación lingüística	19
4.1.1. Tokenización y el algoritmo WordPiece	19
4.1.2. Lematización y normalización lexicológica	20
4.1.3. Modelo de espacio vectorial: TF-IDF	20
4.1.4. Arquitectura Transformer y mecanismo de auto-atención	21
4.1.5. BETO: representaciones bidireccionales para el español	21
4.2. Paradigmas de inferencia y clasificación semántica	22
4.2.1. Clasificación zero-shot por similitud coseno	22
4.3. Agrupamiento semántico y reducción de dimensionalidad	22
4.3.1. UMAP: Proyección Topológica Preservadora de Estructura	22
4.3.2. HDBSCAN: Agrupamiento Jerárquico por Densidad Variable	23
4.3.3. c-TF-IDF: Relevancia de Términos por Clúster	24
4.4. Técnicas de Deduplicación <i>Cross-Site</i>	24
4.4.1. Hash Exacto MD5	24
4.4.2. Similitud de Jaccard sobre Conjuntos de Tokens	24
4.4.3. SimHash: Huellas Digitales Localmente Sensibles	25
4.4.4. Similitud Coseno sobre Vectores TF-IDF	25
4.5. Tecnologías de Extracción de Datos y Persistencia	25
4.5.1. Protocolos de Sindicación: RSS y Atom	25
4.5.2. Evasión de WAF: <i>StealthSession</i> y TLS/JA3 Fingerprinting	25
4.5.3. Propiedades ACID de postgresql	26
4.5.4. Índices GIN y Full-Text Search en Español	26
4.6. Sistemas Neuro-Simbólicos: Fundamentos Teóricos	27
5. Estado del Arte	28
5.1. Sistemas de monitoreo de medios tradicionales	28
5.2. NLP Aplicado a la Detección de Violencia de Género	28
5.3. Herramientas y Esfuerzos Existentes en México	29
5.3.1. Red por los Derechos de la Infancia en México (REDIM)	29
5.3.2. El Mapa de los Feminicidios en México	29
5.3.3. Fundación Futuro con Derechos	29
5.4. Análisis comparativo y brecha identificada	29
5.5. Técnicas de extracción de datos	30
5.5.1. Extracción Estática (HTML Parsing con BeautifulSoup)	30
5.5.2. Automatización de Navegador (Selenium / Playwright)	31
5.5.3. Agregación Estructurada (RSS/Atom) y Stealth Sessions	31
5.5.4. Tabla Comparativa de Técnicas de Extracción	32
5.6. Evolución del Almacenamiento: de Archivos Planos a PostgreSQL	32
5.6.1. Archivos Planos (CSV): Ventajas, Limitaciones y el Techo del TT1	32
5.6.2. PostgreSQL 16: Justificación de la Selección	33
5.7. Evolución de los Modelos de Representación Lingüística para NLP	34
5.7.1. Representaciones Vectoriales Estáticas: Word2Vec y GloVe	34

5.7.2. Modelos Recurrentes: LSTM y GRU	35
5.7.3. Arquitectura Transformer y Atención Bidireccional: BERT	35
5.7.4. Justificación de BETO frente a Modelos Multilingües (mBERT)	35
5.7.5. Clasificación <i>Zero-Shot</i> frente al <i>Fine-Tuning</i> Supervisado	36
5.7.6. Soberanía de Datos y Rechazo de APIs Privativas	36
5.8. Técnicas de Agrupamiento Semántico y Organización Temática	37
5.8.1. Métodos Tradicionales: K-Means y su Inadecuación para Datos de Alta Dimensionalidad	37
5.8.2. DBSCAN: Ventajas y Limitaciones Residuales	37
5.8.3. BERTopic: Arquitectura Modular para Descubrimiento de Tópicos Semánticos	38
5.8.4. Evolución Comparativa TT1 → TT2: de Léxico a Semántica	40
5.9. Conclusión del Capítulo: La Convergencia Tecnológica al Servicio de los Derechos de la Infancia	41
6. Metodología de desarrollo	42
6.1. Metodología de desarrollo evolutivo	42
6.1.1. Cronograma general de evolución del proyecto	42
6.2. Evolución metodológica de los prototipos	43
6.2.1. Fase TT-1 Prototipos exploratorios e ingesta sintáctica (P0-P2)	43
6.2.2. Fase TT-2 Arquitectura de aprendizaje profundo y clasificador escalonado (P3-P4)	44
6.3. Evolución del Stack Tecnológico (TT-1 → TT-2)	44
6.4. Arquitectura de Microservicios, Contenerización y Gestión de Recursos	45
6.4.1. Contenedor de base de datos <i>nna-postgres</i>	45
6.4.2. Contenedor del Backend <i>nna-analyzer</i>	46
6.4.3. Contenedor de interfaz analítica y dashboard web <i>nna-webapp</i>	46
6.5. Especificación de requerimientos	46
7. Desarrollo del Trabajo Terminal I	48
7.1. Prototipo 0: Scraping directo sobre HTML	48
7.1.1. Premisa y Enfoque	48
7.1.2. Resultado: Inviabilidad por seguridad perimetral	48
7.1.3. Decisión Técnica	49
7.2. Prototipo 1: RSS + K-Means	49
7.2.1. Arquitectura implementada	49
7.2.2. Limitaciones Identificadas	49
7.3. Prototipo 2: Triple Fuente + DBSCAN + <i>FeminicideDetector</i>	50
7.3.1. Evolución del sistema	50
7.4. Limitaciones identificadas	50
7.4.1. Falsos positivos: Insuficiencia de la Sintaxis	50
7.4.2. Clustering Léxico: DBSCAN sobre TF-IDF	51
7.4.3. La poca escalabilidad del almacenamiento en CSV	51
7.4.4. Síntesis: La necesidad de comprensión semántica	51
7.5. Diseño de Datos	52
7.5.1. Modelo de Dominio	52
7.5.2. Diccionario de Datos Extendido	52

8. Desarrollo de Trabajo Terminal II	56
8.1. Contexto: Limitaciones del Prototipo 2	56
8.2. Patrones arquitectónicos del flujo analítico	57
8.2.1. Patrón <i>Repository</i> para la capa de persistencia	57
8.2.2. Patrón <i>Pipeline</i> para el flujo analítico	57
8.3. Prototipo 3 Motor semántico BETO y migración de persistencia	57
8.3.1. Migración de CSV a PostgreSQL	58
8.3.2. Arquitectura del motor BETO	58
8.3.3. Clasificación <i>Zero-Shot</i> y <i>Temperature Scaling</i>	59
8.3.4. Evolución del recolector	59
8.3.5. Deduplicación en cascada de cuatro fases	61
8.4. Prototipo 4 Arquitectura híbrida de clasificación	61
8.4.1. Por qué la IA sola no basta	63
8.4.2. Investigador profundo	63
8.5. Sistema de clasificación escalonada	66
8.6. Verificación, Validación y Pruebas (QA)	68
8.6.1. Pruebas Unitarias y Aislamiento de Componentes	68
8.6.2. Pruebas de Caja Negra: Validación del Bozal de Penalización	68
8.6.3. Matriz de Casos de Prueba	68
9. Resultados y Evaluación	70
9.1. Evaluación de la Recolección (Volumen de Ingesta)	70
9.2. Evaluación de la Detección Semántica (Filtro del Embudo)	70
9.3. Evaluación del Agrupamiento (BERTopic)	71
9.4. Análisis de la Interfaz (UI/UX) e Impacto Operativo	71
10. Conclusiones y trabajo a futuro	73
10.1. Conclusión general y cumplimiento de objetivos	73
10.1.1. Cumplimiento del OE-1 Identificación de Fuentes	73
10.1.2. Cumplimiento del OE-2 Módulo de Web Scraping	73
10.1.3. Cumplimiento del OE-3 Procesamiento de Lenguaje Natural (NLP)	74
10.1.4. Cumplimiento del OE-4 Base de Datos y Dashboard Web	74
10.2. Evolución del Prototipo 0 al Prototipo 4	74
10.3. Aportaciones Técnicas y Superación de Retos Críticos (Logros de TT2)	75
10.3.1. Mitigación del “Efecto Péndulo” mediante Arquitectura Híbrida Escalonada	75
10.3.2. Optimización bajo Restricciones Severas de Hardware Local (8GB RAM)	75
10.4. Trabajo Futuro (Líneas de Investigación Abiertas)	76

Índice de figuras

6.1. Cronograma metodológico de evolución y transición de los prototipos del sistema.	43
7.1. Diagrama de Clases / Dominio detallando Tipos de Datos y Relaciones.	53
8.1. Arquitectura del motor BETO y clasificación <i>Zero-Shot</i> con <i>Temperature Scaling</i>	60
8.2. Flujo de deduplicación en cascada.	61
8.3. Flujo de investigación profunda	65
8.4. Arquitectura del Sistema de Clasificación Escalonada	67
9.1. Métricas de filtrado preliminar en el Dashboard Web.	71
9.2. Visualización jerárquica de noticias de Alta Relevancia en el Dashboard.	72

Índice de tablas

1. Información general y acta de constitución del proyecto	vii
2.1. Catálogo de Casos de Uso del Sistema	6
2.2. Requerimientos Funcionales: Subsistema de Recolección	7
2.3. Requerimientos Funcionales: Subsistema de Deduplicación	8
2.4. Requerimientos Funcionales: Subsistema de Análisis Semántico (Motor BETO)	9
2.5. Requerimientos Funcionales: Subsistema de Agrupamiento BERTopic	10
2.6. Requerimientos Funcionales: Subsistema de Persistencia	11
2.7. Requerimientos No Funcionales: Métricas de Calidad y Restricciones	12
3.1. Distinción entre información procesada y excluida	15
3.2. Correspondencia entre objetivos del protocolo y componentes implementados	18
5.1. Comparativa de técnicas de extracción de datos para monitoreo de medios	32
5.2. Comparativa de modelos de representación lingüística para clasificación semántica	36
5.3. Comparativa de algoritmos de agrupamiento semántico para corpus periodístico	40
6.1. Evolución del stack tecnológico	45
6.2. Matriz de trazabilidad general	47
7.1. Comparativa de métricas: TT1 vs. metas del TT2	51
7.2. Diccionario de Datos: Entidad noticias.	52
7.3. Diccionario de Datos: Entidad detecciones.	54
7.4. Diccionario de Datos: Entidad clusters_semanticos.	54
7.5. Diccionario de Datos: Entidad fuentes.	55
7.6. Diccionario de Datos: Entidad usuarios.	55
8.1. Casos de uso del módulo de investigación profunda	66
8.2. Matriz de Casos de Prueba Críticos y Resultados de Ejecución.	69

Project Charter

Proyecto:	2026-A135: Sistema Inteligente para la Identificación y Seguimiento de NNA		
Responsable:	Héctor Alberto Morales Martínez, desarrollador principal		
Autoriza:	Comisión de Trabajos Terminales (CATT)		
Background/Contexto:	Miles de NNA quedan en orfandad por feminicidios anualmente; actualmente no existe un sistema centralizado de seguimiento para apoyarlos y visibilizar su situación.		
Beneficios esperados:	Visibilidad del fenómeno, reducción radical del trabajo manual de monitoreo para ONGs y estructuración de una base de datos histórica mediante PNL.		
Costo estimado:	Proyecto Académico de Investigación (TT1/TT2)		
Fecha de inicio:	Septiembre 2025	Fecha de término:	Junio 2026
Objetivo:	Desarrollar un sistema automatizado e inteligente de recolección, análisis PNL y visualización de eventos de orfandad por feminicidio en México.		
Entregables Principales			
	SIST-01	Sistema web orquestado en microservicios contene- rizados (Docker Compose)	
	DOC-01	Documentación Técnica (este documento)	
	COD-01	Código fuente modular en repositorio	
Alcance del proyecto			
Incluye:	• Recolección multi-fuente con mas de 50 feeds RSS y Google News API. • Clasificación semántica con el modelo BETO Fine-tuned localmente. • Eliminación de ruido con clasificación escalonada. • Modelado automático de tópicos con BERTopic. • Persistencia con PostgreSQL y dashboard web con modulo de investigación profunda.		
Excluye:	• Identificación, almacenamiento o filtrado de datos personales completos y sensibles de NNA. • Intervención social, apoyo psicológico directo o canalización jurídica oficial ante autoridades. • Validación pericial o legal de la veracidad jurídica de los hechos publicados por los medios.		
Metodología:	Desarrollo evolutivo por prototipos (P0 a P4).		
Datos de contacto			
Desarrollador:	Héctor Alberto Morales Martínez		
Directores:	M. en C. Ulises Saldaña Velez, M. en C. Gabriel Hurtado Avilés		
Organización de Apoyo:			
Riesgos y peligros:	• Bloqueos anti-scraping progresivos y cambios en la estructura de los portales web.		
Supuestos:	• Las fuentes periodísticas locales mantienen un marco de narrativa policiaca para crímenes de género. • Los servicios y APIs de agregación semántica de Google News permanecen accesibles.		

Tabla 1: Información general y acta de constitución del proyecto

0.1. Glosario y Conceptos Técnicos del Sistema

En esta sección se definen los conceptos de ingeniería de software, aprendizaje profundo, seguridad y operación legal que fundamentan la arquitectura del sistema.

Active Learning (Aprendizaje Activo): Estrategia de entrenamiento interactivo donde el modelo de machine learning selecciona y solicita iterativamente el etiquetado de los ejemplos más informativos o confusos (*Hard Negatives*), acelerando la convergencia y mejorando la precisión con un volumen reducido de datos.

BETO: Modelo de lenguaje masivo preentrenado basado en la arquitectura *BERT* bidireccional, especializado en la sintaxis y semántica del idioma español. Actúa como el motor principal de representación relacional en el sistema.

BERTopic: Algoritmo neuro-estadístico para el modelado dinámico de tópicos. Se alimenta de los tensores de *Transformers* y aplica reducción de dimensionalidad para crear agrupaciones semánticas del corpus de noticias.

Deduplicación Estocástica Multidimensional: Pipeline de limpieza de datos en 4 capas (MD5 → SimHash → Jaccard → Coseno TF-IDF) diseñado para identificar y unificar noticias redundantes publicadas por diferentes medios de comunicación, garantizando la integridad de los eventos fácticos.

Efecto Péndulo: Fenómeno de colisión semántica diagnosticado en el Prototipo 3, donde el modelo Transformer exhibía falsos positivos al confundir textos legislativos (ej. Ley Monzón) o informes estadísticos con crímenes reales debido al alto solapamiento de tokens léxicos (“víctima”, “menor”, “homicidio”).

Feminicidio: Muerte violenta de las mujeres por razones de género, tipificada legalmente en el sistema penal mexicano. Constituye el evento desencadenante principal que la plataforma rastrea de forma automatizada.

Fine-Tuning (Ajuste Fino Supervisado): Proceso algorítmico mediante el cual los pesos sinápticos de un modelo preentrenado masivo (BETO) se actualizan localmente utilizando un dataset específico de dominio (noticias de nota roja) para especializar sus predicciones en inferencia binaria.

GIN Index (Generalized Inverted Index): Estructura de indexación algorítmica desplegada en PostgreSQL 16. Optimiza transversalmente las consultas relacionales para Búsqueda de Texto Completo (*Full-Text Search*) sobre grandes corpus de noticias.

Gradient Accumulation (Acumulación de Gradientes): Técnica de ingeniería en aprendizaje profundo para entrenar redes masivas en hardware con memoria limitada (8 GB de RAM). Consiste en acumular los gradientes de varios lotes pequeños durante los pases hacia adelante antes de efectuar una actualización real de los pesos sinápticos, emulando un tamaño de lote (*batch size*) virtual mayor sin provocar errores de *Out Of Memory*.

HDBSCAN: Algoritmo de agrupamiento espacial basado en densidad jerárquica. Integrado en BERTopic para aislar matemáticamente clústeres de noticias descartando proactivamente anomalías (*outliers*) como ruido estadístico.

JA3 Fingerprinting: Método de creación de firmas criptográficas basado en la forma en que una aplicación cliente (como un navegador) inicia un apretón de manos (*handshake*) TLS.

Linear Probing (Sondeo Lineal): Técnica de especialización de Transformers que consiste en congelar de manera estricta los pesos de las capas internas (*encoder*) y entrenar exclusivamente la última capa densa de clasificación superior, limitando el consumo de memoria volátil y previniendo el desbordamiento local.

NNA: Acrónimo demográfico de Niñas, Niños y Adolescentes. Objeto focal de la extracción semántica para la identificación de infancias en orfandad.

Scoring Escalonado v7.1 (Tiered Scoring): Arquitectura neuro-simbólica híbrida que evalúa la confianza de una noticia a través de 4 capas concurrentes de decisión. Sustituye la lógica de exclusión absoluta (vetos) por penalizaciones temáticas suaves ($-0,35$) y bonificaciones heurísticas sintácticas, garantizando una alta tasa de precisión (*F1-Score*).

StealthSession: Clase de programación estructurada que administra las conexiones HTTP, emulando huellas digitales de red para eludir las mitigaciones antibot de los firewalls de aplicación web (WAF) durante el raspado de datos asíncrono.

UMAP: Herramienta de aprendizaje topológico empleada para comprimir drásticamente la representación de alta dimensionalidad (768 dimensiones) de BETO, agilizando el agrupamiento espacial de textos relacionados.

Víctima Indirecta: Entidad legal referida a las personas dependientes económicamente o con lazos consanguíneos de primer grado hacia la víctima de un feminicidio. El núcleo del sistema radica en su detección programática.

El presente Trabajo Terminal registrado bajo el número de protocolo 2026-A135, constituye la segunda fase de un proyecto de investigación aplicada que aborda una problemática crítica en la intersección de los derechos humanos, la violencia de género y la ingeniería de sistemas inteligentes: la invisibilidad estadística y la ausencia de seguimiento sistemático de las Niñas, Niños y Adolescentes (NNA) en situación de orfandad por feminicidio en México. Mediante la conceptualización, diseño, desarrollo y despliegue de un sistema inteligente de monitoreo de medios, el proyecto propone una solución tecnológica para mitigar la dispersión de datos que obstaculiza la labor de las organizaciones civiles y de las instituciones gubernamentales en la procuración de justicia y la reparación integral del daño.

1.1. Presentación del proyecto

México atraviesa una crisis sistémica de violencia feminicida de proporciones documentadas. De acuerdo con los registros del Secretariado Ejecutivo del Sistema Nacional de Seguridad Pública (SESNSP) y la Secretaría de Gobernación, entre los años 2010 y 2023 se han contabilizado **más de 8,000 víctimas** de feminicidio en el territorio nacional. Esta cifra ya de por sí alarmante, oculta una dimensión adicional del daño: el impacto colateral sobre los hijos e hijas de las víctimas, quienes quedan en situación de orfandad repentina y traumática.

La Red por los Derechos de la Infancia en México (REDIM) estima que existen **al menos 3,500 Niñas, Niños y Adolescentes** que han perdido a su madre como consecuencia directa de un feminicidio. Pese a la gravedad de esta cifra no existe a nivel nacional un padrón oficial, un censo unificado ni un mecanismo de seguimiento estandarizado que documente el estado de estas víctimas indirectas. La Convención sobre los Derechos del Niño y la Ley General de los Derechos de Niñas, Niños y Adolescentes (LGDNNA) establecen el principio del *interés superior de la niñez* como eje que orienta, garantizando el derecho a la protección, la salud, la educación y la reparación integral. No obstante la ausencia de datos estructurados imposibilita la materialización efectiva de estos derechos.

El TT no. 2026-A135 surge como respuesta tecnológica a esta brecha. El sistema automatiza el ciclo completo de descubrimiento informativo desde la recolección de noticias en fuentes periodísticas de acceso público hasta la clasificación semántica y la persistencia relacional de los casos identificados, esto con el objetivo de que ningún NNA en situación de orfandad por feminicidio permanezca invisible para las instituciones que tienen la obligación legal de protegerlo.

1.2. Planteamiento del problema

La información sobre los NNA en situación de orfandad por feminicidio se encuentra fragmentada en el ecosistema informativo mexicano. Los datos relevantes sobre la existencia, el estado legal y las condiciones de resguardo de los hijos de una víctima suelen estar dispersos en notas periodísticas de medios locales, comunicados aislados de fiscalías estatales y registros internos de organizaciones no gubernamentales que no se interconectan entre sí. La información existe pero no es accesible de forma sistemática ni procesable a escala computacional.

1.2.1. La inviabilidad del monitoreo manual

La mitigación de esta invisibilidad estadística recae actualmente sobre organizaciones de la sociedad civil como la Fundación "Futuro con Derechos".^{el} proyecto cartográfico de la investigadora María Salguero y la REDIM. La metodología subyacente a este trabajo de documentación presenta un cuello de botella operativo: la **revisión manual y artesanal de fuentes hemerográficas**. El proceso exige que analistas y voluntarios dediquen muchas horas semanales a la revisión individual de portales de noticias, buscando menciones colaterales de hijos en narrativas de notas policíacas. Este enfoque es inviable frente al volumen de información que se genera diariamente en el país ya que México cuenta con más de 1,200 medios periodísticos digitales activos, produciendo decenas de miles de artículos diarios.

La latencia introducida por el factor humano limita drásticamente la capacidad de procesamiento, provocando que una enorme cantidad de casos pasen desapercibidos porque es imposible auditar concurrentemente todas las fuentes periodísticas nacionales y regionales. Además este proceso consume los recursos de talento especializado (psicólogos, abogados, trabajadores sociales), desviando su capacidad hacia labores de recolección de datos en lugar de la intervención directa con las víctimas.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Desarrollar un sistema inteligente que sea capaz de automatizar la recolección, clasificación y análisis de información relacionadas con casos de feminicidio, para facilitar la identificación de niños, niñas y adolescentes que se encuentren en una situación de orfandad en consecuencia de los crímenes de violencias. Mediante el uso de Web Scraping y Procesamiento de Lenguaje Natural

1.3.2. Objetivos Específicos

- OE-1** Identificar y seleccionar fuentes públicas confiables que contengan información relacionada con feminicidios y posibles víctimas indirectas.
- OE-2** Diseñar y desarrollar una herramienta automatizada de Web Scraping para la recolección sistemática de datos provenientes de dichas fuentes.
- OE-3** Implementar técnicas de Procesamiento de Lenguaje Natural (PLN) para extraer y estructurar información clave a partir de textos no estructurados.
- OE-4** Diseñar una base de datos estructurada que permita almacenar la información recolectada de manera eficiente y segura.

Justificación y Alcance del Proyecto

En este capítulo se presenta la justificación social, tecnológica y ética que sustenta el desarrollo del *Sistema Inteligente para la Identificación y Seguimiento de NNA* y se delimita el alcance del sistema, sus limitaciones y la alineación con el protocolo de Trabajo Terminal 2026-A135.

2.1. Justificación Social

Entre 2010 y 2023 más de 8,000 mujeres fueron víctimas de feminicidio en México, según datos de la Secretaría de Gobernación [17]. Detrás de estas cifras se encuentran miles de niñas, niños y adolescentes (NNA) que quedaron en condición de orfandad. La Red por los Derechos de la Infancia en México (REDIM) estima que al menos 3,500 NNA perdieron a su madre por feminicidio entre 2012 y 2022 [15], sin embargo no existe un censo oficial ni un sistema de seguimiento que permita dimensionar con precisión el alcance de esta crisis.

La información sobre los hijos e hijas de las víctimas se encuentra fragmentada y dispersa pues puede aparecer en una nota periodística local, en el comunicado de una fiscalía estatal o en los archivos internos de alguna organización civil. Sin un mecanismo que conecte estos fragmentos de información resulta prácticamente imposible rastrear qué ha sucedido con esos menores y de si recibieron atención psicológica, si mantienen acceso a educación, si se les garantizó reparación del daño o si fueron canalizados a instituciones como en el DIF.

Algunas de las consecuencias de esta dispersión son:

- **Para las organizaciones civiles:** Organizaciones como *Futuro con Derechos* dedican varias horas semanales a revisar manualmente decenas de sitios de noticias buscando casos donde se mencione a hijos de víctimas de feminicidio. Reconocen que muchos casos pasan desapercibidos porque es imposible revisar todas las fuentes disponibles con recursos humanos limitados.
- **Para las autoridades:** Sin datos estructurados ni trazables las instituciones gubernamentales carecen de información para evaluar la efectividad de sus programas de atención a víctimas indirectas y para diseñar políticas públicas basadas en evidencia.
- **Para las familias:** Los familiares de las víctimas frecuentemente desconocen sus derechos, las instituciones de apoyo disponibles y los mecanismos de reparación del daño. La falta de un sistema de seguimiento dificulta la coordinación interinstitucional.

Un sistema automatizado que actúe como filtro inicial que procese cientos de noticias, identificando así las potencialmente relevantes y presentándolas estructuradas, permite que las organizaciones dediquen su tiempo al seguimiento de casos concretos en lugar de a la búsqueda manual de información.

2.2. Justificación Tecnológica

La revisión manual de medios de comunicación no es viable a escala nacional pues México cuenta con cientos de medios digitales (nacionales, estatales y locales) que publican diariamente noticias relacionadas con feminicidios. La cobertura periodística de cada caso varía significativamente pues algunos eventos son cubiertos por múltiples medios con redacciones diferentes, mientras que otros aparecen exclusivamente en medios regionales de bajo alcance.

Durante el desarrollo del Trabajo Terminal I, se probaron tres enfoques de recolección que evidenciaron la complejidad técnica del problema:

1. **Scraping directo (Prototipo 0):** Fracasó completamente. Los medios modernos emplean protecciones como cloudflare, contenido renderizado con JavaScript y bloqueos HTTP 403, haciendo complicada la extracción directa del HTML.
2. **RSS básico (Prototipo 1):** Funcionó parcialmente con 8 fuentes RSS, recolectando aproximadamente 56 noticias en dos semanas. La cuestión era que la cobertura se limitaba a medios nacionales con feeds públicos.
3. **Triple fuente (Prototipo 2):** Combinó 10 RSS, Google News y scraping histórico, alcanzando 250 noticias únicas. Este enfoque nos hizo ver que la viabilidad de la recolección multi-fuente existía, pero aún mantenía limitaciones en la detección semántica (Mas del 80 % de falsos positivos con expresiones regulares).

Trabajo Terminal II representa una evolución significativa de estas estrategias mediante la integración de tecnologías de procesamiento de lenguaje natural (PLN):

BETO (BERT en español): Modelo de lenguaje preentrenado con 110 millones de parámetros que permite comprender el contexto semántico de las noticias, distinguiendo entre casos concretos de feminicidio con NNA afectados y noticias de legislación, campañas o estadísticas que mencionan las mismas palabras clave.

BERTopic: Sistema de clustering semántico que combina embeddings contextuales (BETO), reducción de dimensionalidad (UMAP) y clustering basado en densidad (HDBSCAN) para agrupar noticias por similitud de significado, no solo de vocabulario.

Deduplicación: Sistema de cuatro algoritmos (Hash MD5, Jaccard, SimHash, TF-IDF coseno) que elimina automáticamente noticias duplicadas o cuasi-duplicadas provenientes de diferentes medios.

PostgreSQL: Base de datos relacional que reemplaza el almacenamiento en archivos CSV lo que habilita consultas complejas, integridad referencial y acceso concurrente.

Gracias a estas tecnologías se transforma texto no estructurado en datos consultables, permitiendo análisis cuantitativos (tendencias temporales, distribución geográfica) y cualitativos (agrupación temática, seguimiento de casos) que serían imposibles mediante revisión manual.

2.3. Justificación ética y moral: Protección de la privacidad de los NNA

2.3.1. El interés superior del niño

El desarrollo de este sistema se enmarca en un principio ético fundamental: **el interés superior del niño**, establecido en el artículo 3 de la Convención sobre los Derechos del Niño de las Naciones Unidas [13] y en el artículo 2 de la Ley General de los Derechos de Niñas, Niños y Adolescentes de México [3]. Este principio establece que cualquier acción que involucre a menores de edad debe priorizar su bienestar, protección y dignidad por encima de cualquier otro interés.

En base a este principio, el sistema ha sido diseñado desde su inicio con una restricción no negociable: **priorizar el manejo cuidadoso y confidencial de la información obtenida de los NNA.**

2.3.2. Naturaleza de la información procesada

Es importante aclarar la naturaleza de los datos que el sistema procesa y la distinción entre lo que el sistema *identifica* y lo que *no identifica*:

Tabla 2.1: Distinción entre información procesada y excluida

El sistema SÍ identifica	El sistema NO identifica
Noticias que mencionan feminicidios	Nombres completos de los NNA afectados
Menciones genéricas a NNA como víctimas indirectas (ej. "dejó 3 hijos menores")	Datos personales de los menores (edad exacta, CURP, domicilio)
Contexto del caso: ubicación geográfica, fecha, instituciones involucradas	Identidad verificada de ningún menor
Nombres de víctimas adultas (cuando aparecen en la nota periodística)	Fotografías ni datos biométricos de NNA
Patrones y tendencias estadísticas	Información no publicada en medios

2.3.3. ¿Por qué el sistema no proporciona nombres de NNA?

La ausencia de nombres e identidades de los NNA en los resultados del sistema **no es una limitación técnica, sino una decisión ética respaldada por un marco normativo y periodístico sólido:**

1. **Los medios de comunicación protegen la identidad de los menores.** El Código de Ética Periodística de la UNESCO y los códigos editoriales de los principales medios mexicanos establecen que la identidad de los NNA víctimas —directas o indirectas— debe ser resguardada. Por esta razón, las notas periodísticas que constituyen la fuente primaria del sistema rara vez incluyen los nombres completos de los menores. En su lugar, utilizan referencias genéricas: "sus tres hijos menores de edad", "una niña de 5 años", "los menores quedaron bajo custodia de la abuela". Esta práctica periodística responsable se refleja necesariamente en los datos que el sistema puede extraer.
2. **La Ley General de los Derechos de Niñas, Niños y Adolescentes (LGDNNA).** El artículo 76 de la LGDNNA prohíbe la difusión de datos personales de NNA que permitan su identificación cuando estos se encuentran en situación de vulnerabilidad. Los NNA que han perdido a su madre por feminicidio se encuentran, por definición, en una de las situaciones de mayor vulnerabilidad contempladas por la ley. Cualquier sistema que pretendiera identificarlos nominalmente estaría en violación directa de esta disposición legal.

3. **El Protocolo de Palermo y la normativa internacional.** Los instrumentos internacionales de protección a la infancia, incluido el Protocolo Facultativo de la Convención sobre los Derechos del Niño, establecen que los Estados deben garantizar la privacidad de los menores víctimas en todos los procedimientos, incluidos los de documentación e investigación. Nuestro sistema respeta este marco al limitarse a identificar *la existencia* de NNA afectados, sin revelar *quiénes son*.
4. **Principio de minimización de datos (RGPD/LFPDPPP).** Siguiendo las mejores prácticas internacionales de protección de datos —reflejadas en el Reglamento General de Protección de Datos de la Unión Europea y en la Ley Federal de Protección de Datos Personales en Posesión de Particulares de México— el sistema recolecta y almacena únicamente los datos estrictamente necesarios para cumplir su propósito: identificar *la existencia de noticias* sobre NNA afectados, no la identidad de los NNA mismos.

2.3.4. Fortaleza ética del enfoque adoptado

Lejos de ser una debilidad, esta restricción constituye una **fortaleza ética fundamental** del sistema:

- **El sistema identifica el fenómeno, no a las personas.** Su valor radica en hacer visible la dimensión de la crisis de orfandad por feminicidio —cuántos casos, dónde, con qué frecuencia— sin comprometer la dignidad ni la privacidad de los menores involucrados.
- **Herramienta de apoyo, no sustituto de intervención.** El sistema está concebido como un instrumento de alerta temprana y documentación que facilita el trabajo de organizaciones especializadas. La intervención social, la verificación de datos y el contacto con las familias corresponden exclusivamente a profesionales capacitados en trabajo social, psicología y derecho, quienes operan bajo protocolos estrictos de confidencialidad.
- **Alineación con IA ética.** El diseño del sistema se adhiere a los principios de Inteligencia Artificial ética establecidos por la OECD [?]: transparencia en el procesamiento, responsabilidad en el uso de datos, y priorización del bienestar humano sobre la capacidad técnica.

2.3.5. Compromiso de privacidad

El equipo de desarrollo establece el siguiente compromiso formal:

El Sistema Inteligente para la Identificación y Seguimiento de NNA se compromete a no recolectar, almacenar, procesar ni difundir datos personales de niñas, niños y adolescentes. Toda la información procesada proviene exclusivamente de fuentes periodísticas públicas, y los resultados del sistema se limitan a identificar la existencia de noticias relevantes, patrones estadísticos y tendencias temporales, sin revelar la identidad de ningún menor de edad.

2.4. Alcance del Sistema

2.4.1. Alcance funcional

El sistema desarrollado abarca las siguientes capacidades, implementadas progresivamente a lo largo de cinco prototipos (P0–P4):

- **Recolección automatizada multi-fuente:** Extracción de noticias desde aproximadamente 45 fuentes RSS, 14 queries de Google News, scraping dinámico de HTML/Sitemap y búsquedas en Google Search con StealthSession.
- **Detección semántica de relevancia:** Clasificación de noticias mediante un scoring de 4 ejes (feminicidio, NNA, caso individual, víctima indirecta) utilizando el modelo BETO (BERT en español) con inferencia por lotes (batch inference) en PyTorch.

- **Deduplicación cross-site:** Eliminación automática de noticias duplicadas o cuasi-duplicadas mediante cascada de 4 algoritmos (Hash MD5, Jaccard, SimHash, TF-IDF coseno).
- **Clustering semántico:** Agrupación de noticias por similitud de significado mediante BERTopic (BETO embeddings + UMAP + HDBSCAN + c-TF-IDF).
- **Almacenamiento relacional:** Persistencia en PostgreSQL con esquema normalizado, Full-Text Search e índices optimizados.
- **Dashboard interactivo:** Interfaz web con autenticación (Argon2id), gestión de fuentes, filtros avanzados, visualización de estadísticas con Chart.js y exportación de datos.
- **Gestión de fuentes:** Módulo para agregar, editar, probar, sondear y activar/desactivar fuentes de noticias directamente desde la interfaz web.

2.4.2. Alcance geográfico y temporal

Geográfico: Cobertura nacional con énfasis en medios mexicanos. Incluye medios nacionales (La Jornada, Proceso, Animal Político, El Universal, Milenio, entre otros), medios especializados en género y derechos humanos (CIMAC Noticias, Luchadoras) y medios regionales (El Sol de Toluca, Diario de Xalapa, Noroeste, entre otros).

Temporal: El sistema procesa noticias desde enero de 2025 hasta la fecha actual (mayo 2026), con capacidad de extensión retrospectiva mediante el módulo de scraping histórico.

2.5. Limitaciones y exclusiones

2.5.1. Exclusiones explícitas del alcance

El sistema **no** contempla las siguientes funcionalidades:

- No identifica ni almacena nombres ni datos personales de NNA reales (justificación ética detallada en la Sección 3.3).
- No realiza intervención social directa. Es una herramienta de documentación y alerta, no un sistema de gestión de casos.
- No accede a bases de datos gubernamentales cerradas (fiscalías, DIF, registros civiles). Toda la información proviene de fuentes periodísticas públicas.
- No valida la veracidad de las noticias. El sistema asume que las fuentes periodísticas monitoreadas mantienen estándares editoriales mínimos pero no verifica la veracidad de cada nota.
- No implementa Named Entity Recognition (NER) completo con fine-tuning. Esta funcionalidad se identifica como trabajo futuro.

2.5.2. Limitaciones técnicas

1. **Dependencia de fuentes periodísticas:** La cobertura del sistema está limitada a lo que los medios publican. Feminicidios no cubiertos por la prensa o que ocurren en zonas con escasa cobertura mediática no serán detectados.
2. **Precisión del modelo semántico:** Aunque BETO mejora sustancialmente la detección respecto a los patrones regex del TT1, el modelo de zero-shot classification tiene limitaciones inherentes al no estar fine-tuned específicamente para el dominio. Se estima una tasa de falsos positivos $\leq 5\%$.
3. **Protecciones anti-scraping:** Algunos medios implementan protecciones agresivas (CAPTCHAs, rate limiting estricto, paywalls) que pueden impedir la recolección de sus contenidos.

4. **Latencia en tiempo real:** El sistema opera con ciclos de recolección programados (cada 6–24 horas), no en tiempo real. Existe un desfase entre la publicación de una noticia y su procesamiento.
5. **Idioma:** El sistema está optimizado exclusivamente para noticias en español. Noticias en lenguas indígenas o en inglés no son procesadas.

2.6. Alineación con el protocolo

El desarrollo del trabajo terminal se alinea con las metas y entregables definidos en el protocolo no. 2026-A135, aprobado por la comisión de trabajos terminales. La siguiente tabla muestra la correspondencia entre los objetivos del protocolo y los componentes implementados:

Tabla 2.2: Correspondencia entre objetivos del protocolo y componentes implementados

Meta del Protocolo	Componente Implementado	Estado
Migrar detección de regex a modelos semánticos	SemanticDetector con BETO (zero-shot + 4 ejes)	✓
Implementar base de datos relacional	PostgreSQL con SQLAlchemy ORM	✓
Ampliar cobertura de fuentes	45+ fuentes RSS + Google Search + scraping dinámico	✓
Implementar clustering semántico	BERTopic (BETO + UMAP + HDBSCAN)	✓
Desarrollar interfaz web completa	Dashboard con autenticación, gestión de fuentes, filtros	✓
Deduplicación avanzada	Cascada de 4 fases (Hash, Jaccard, SimHash, TF-IDF)	✓
Implementar NER completo	Parcial: extracción contextual básica	~

Nota: La implementación de Named Entity Recognition (NER) con fine-tuning de spaCy, originalmente contemplada en el protocolo, se implementó de forma parcial mediante extracción contextual basada en patrones. La versión completa con modelos fine-tuned se identifica como trabajo futuro, justificado por la priorización de la precisión del detector semántico y la estabilización de la arquitectura de producción.

En este capítulo se establece el fundamento matemático y conceptual que sustenta la arquitectura neuro-simbólica del Sistema Inteligente para la Identificación y Seguimiento de NNA. Cada sección describe una tecnología implementada en el sistema y justifica su adopción frente a las alternativas disponibles, trazando así una línea directa entre los principios teóricos formales y las decisiones de implementación documentadas.

3.1. Procesamiento de lenguaje natural y representación lingüística

El Procesamiento de lenguaje natural (PLN) comprende el conjunto de técnicas computacionales orientadas a la comprensión, generación y manipulación del lenguaje humano en formato textual. El dominio específico de este proyecto; que son noticias periodísticas mexicanas en español sobre violencia feminicida, exige herramientas de PLN que capturen la semántica contextual del lenguaje, trascendiendo la mera relación de palabras.

3.1.1. Tokenización y el algoritmo WordPiece

La tokenización es el proceso mediante el cual una cadena de texto se divide en unidades elementales de representación (*tokens*) que el modelo puede procesar. Los enfoques clásicos segmentan el texto a nivel de palabra completa lo que genera vocabularios de tamaño inmanejable (más de 500,000 tipos) y deja fuera de vocabulario cualquier término no visto durante el entrenamiento.

El modelo BETO al igual que toda la familia BERT, emplea el algoritmo **WordPiece** [?], el cual es un esquema de segmentación sub-léxica que descompone las palabras en secuencias de sub-unidades frecuentes. El algoritmo construye el vocabulario de forma incremental: inicializa el conjunto con todos los caracteres individuales del corpus y fusiona iterativamente los pares de sub-unidades que maximizan la verosimilitud del corpus de entrenamiento bajo un modelo de lenguaje:

$$\text{Puntuación}(u_1, u_2) = \frac{P(u_1 u_2)}{P(u_1) \cdot P(u_2)} \quad (3.1)$$

Donde u_1 y u_2 son sub-unidades adyacentes y $P(\cdot)$ denota la probabilidad del sub-token en el corpus. Las fusiones con mayor puntuación se aplican repetidamente hasta alcanzar el tamaño de vocabulario objetivo (30,000 tokens para

BETO). Los sub-tokens que continúan una palabra se prefijan con ## para distinguirlos de tokens iniciales, por ejemplo: "feminicidio" se tokeniza como fem##ini##cid##io.

Este esquema tiene dos consecuencias técnicas que son fundamentales para el dominio del sistema. Primero, las palabras desconocidas como neologismos o nombres propios se descomponen en sub-tokens conocidos en lugar de ser descartadas, para así preservar parcialmente su información. Segundo, el entrenamiento con *Whole Word Masking* (WWM) garantiza que todos los sub-tokens de una misma palabra se enmascaren conjuntamente, forzando al modelo a aprender representaciones semánticamente coherentes de términos especializados de múltiples sub-unidades.

Además BETO utiliza dos tokens especiales de estructura: [CLS] (primer token de toda secuencia, cuyo vector de salida representa a la secuencia completa para tareas como clasificación) y [SEP] (que es mas como un separador entre dos segmentos de texto distintos).

3.1.2. Lematización y normalización lexicológica

La lematización es el proceso de reducir una forma flexionada de una palabra a su lema canónico (forma base del diccionario), considerando el contexto morfosintáctico. Se distingue de la *stemming* en que produce formas lingüísticamente válidas: "feminicidios" → "feminicidio" (lematización), no "feminicid" (stemming).

En el sistema, la lematización se emplea en la capa de preprocesamiento del módulo de búsqueda por sinónimos (SynonymDictionary) y en el vectorizador TF-IDF de deduplicación, donde la normalización morfológica reduce la dispersión léxica del vocabulario y mejora la robustez de las métricas de similitud ante variaciones de tiempo verbal, número y género gramaticales comunes en el periodismo mexicano. PostgreSQL aplica su propio proceso de lematización para español (diccionario Snowball) al momento de construir el *tsvector* para la búsqueda de texto completo, cubriendo el caso de recuperación de información en el dashboard.

3.1.3. Modelo de espacio vectorial: TF-IDF

La representación vectorial clásica de documentos de texto se articula mediante el modelo *Term Frequency - Inverse Document Frequency* (TF-IDF) [?]. El peso asignado a un término t en un documento d dentro de una colección D se calcula como el producto de dos factores:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (3.2)$$

Donde la frecuencia de término $\text{TF}(t, d)$ mide la proporción de ocurrencias del término en el documento:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (3.3)$$

Y la frecuencia inversa de documento $\text{IDF}(t, D)$ penaliza los términos ubicuos en la colección (que aportan poca información discriminativa) y premia los términos raros:

$$\text{IDF}(t, D) = \log \frac{|D|}{|\{d \in D : t \in d\}| + 1} \quad (3.4)$$

Este sistema implementa una variante de TF-IDF que multiplica el peso de términos del glosario de dominio (por ejemplo: "orfandad", "DIF", "resguardo") por un factor amplificador $\beta > 1$ que incrementa artificialmente su IDF efectivo. Esta modificación operacionaliza el conocimiento experto del dominio en el espacio vectorial sin requerir reentrenamiento. TF-IDF se emplea en el sistema exclusivamente para las fases de deduplicación y el módulo de sinónimos; la clasificación de relevancia utiliza exclusivamente los embeddings BETO.

3.1.4. Arquitectura Transformer y mecanismo de auto-atención

La arquitectura *Transformer* [18] constituye el fundamento de todos los modelos de lenguaje de última generación. A diferencia de las Redes Neuronales Recurrentes (RNN) y las LSTM, que procesan la secuencia de tokens de forma estrictamente lineal, generando un estado oculto h_t en función del estado h_{t-1} y el token actual, el Transformer prescinde de la recurrencia. Todas las posiciones de la secuencia se procesan en paralelo mediante el mecanismo de auto-atención multi-cabeza (*Multi-Head Self-Attention*).

Para cada token x_i de la secuencia, el mecanismo de atención computa tres vectores: una consulta $Q_i = x_i W^Q$, una clave $K_i = x_i W^K$ y un valor $V_i = x_i W^V$, donde $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ son matrices de proyección aprendidas. La relevancia del token j para el token i se calcula como el producto punto de sus vectores de consulta y clave, escalado para evitar saturación del gradiente en la función softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3.5)$$

Donde d_k es la dimensión de las claves. La salida para el token i es una suma ponderada de todos los vectores V_j de la secuencia, con pesos determinados por la similitud de atención. Esto permite que "menor.^{en} la posición i reciba contribuciones directas de resguardo del DIF.^{en} la posición j , sin importar la distancia lineal entre ambas posiciones.

La auto-atención multi-cabeza ejecuta h instancias paralelas del mecanismo de atención, cada una con sus propias proyecciones W_i^Q, W_i^K, W_i^V , y concatena sus salidas.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (3.6)$$

Donde cada cabeza captura un tipo diferente de relación semántica o sintáctica. BERT-base emplea $h = 12$ cabezas en cada una de sus 12 capas de transformación.

3.1.5. BETO: representaciones bidireccionales para el español

BERT (*Bidirectional Encoder Representations from Transformers*) [4] extiende la arquitectura Transformer original mediante el preentrenamiento bidireccional: en lugar de predecir el siguiente token (modelado de lenguaje causal como GPT) BERT se entrena con el objetivo de **Modelado de Lenguaje Enmascarado** (MLM). En cada secuencia de entrenamiento el 15 % de los tokens se enmascara aleatoriamente y el modelo debe predecir el token original usando el contexto *bidireccional completo* (los tokens a la izquierda y a la derecha de la máscara). Este objetivo fuerza al modelo a desarrollar representaciones profundamente contextuales.

BETO [2] (*bert-base-spanish-wwm-cased*) es la instancia monolingüe de BERT preentrenada exclusivamente en español, con la siguiente arquitectura base:

- 12 capas Transformer (encoders)
- 12 cabezas de atención por capa
- Dimensión del espacio oculto: $d_{\text{model}} = 768$
- 110 millones de parámetros entrenables
- Vocabulario WordPiece de 30,000 tokens en español

El corpus de preentrenamiento incluye wikipedia en español (aproximadamente 3 GB), OpenSubtitles, ParaCrawl, MultiUN (corpus OPUS) y fuentes de prensa latinoamericana. Esta composición es crítica para el dominio del sistema pues el periodismo latinoamericano posee un estilo de redacción y un léxico que difieren del español peninsular. Los corpus de prensa incluidos en el preentrenamiento de BETO sesgan sus representaciones hacia el vocabulario y las

construcciones sintácticas empleadas por los medios de comunicación mexicanos, lo cual mejora la calidad de los embeddings para el dominio específico del sistema.

La representación semántica de una noticia se extrae del *hidden state* del token especial [CLS] en la última capa del encoder que es un vector de 768 dimensiones que condensa el significado global de la secuencia completa. Este vector se normaliza mediante norma L2 para habilitar la comparación por similitud coseno:

$$\hat{e} = \frac{e}{\|e\|_2}, \quad e = \text{BETO}(\text{texto})[0, :] \quad (3.7)$$

Donde $e \in \mathbb{R}^{768}$ es el embedding crudo del token [CLS] y $\hat{e}$ es su versión normalizada empleada en la clasificación zero-shot.

3.2. Paradigmas de inferencia y clasificación semántica

3.2.1. Clasificación zero-shot por similitud coseno

El paradigma de **clasificación zero-shot** [?] permite categorizar documentos en clases definidas en lenguaje natural sin que disponga de ejemplos etiquetados de entrenamiento para dichas clases. Esta implementación adoptada en el sistema es la clasificación por similitud coseno entre embeddings. Se precomputan representaciones BETO para descripciones textuales de las categorías de clasificación ("*noticia que reporta un caso concreto de feminicidio donde los hijos de la víctima quedaron en orfandad o bajo resguardo institucional*" para la clase *relevante*"; "*notas estadísticas, políticas públicas, reporte donde el menor es el agresor o feminicidio de una menor de edad*" para la clase *"no relevante"*).

En tiempo de inferencia, el embedding del texto $\hat{e}_{\text{texto}}$ se compara contra los embeddings de cada categoría $\hat{e}_c$ mediante el producto punto (lo que equivale a la similitud coseno):

$$s_c = \hat{e}_{\text{texto}} \cdot \hat{e}_c = \cos(\hat{e}_{\text{texto}}, \hat{e}_c) \quad (3.8)$$

La mayor ventaja de este paradigma sobre el fine-tuning supervisado es que el conocimiento experto del dominio se codifica directamente en las descripciones de categorías en lenguaje natural, lo cual hace que sean auditables y modificables por cualquier miembro del equipo sin necesidad de reentrenar el modelo. Por otro lado, el modelo neuronal aporta la capacidad de la generalización semántica aprendida durante el preentrenamiento.

3.3. Agrupamiento semántico y reducción de dimensionalidad

El agrupamiento semántico de documentos periodísticos tiene como objetivo identificar grupos de noticias que pertenecen al mismo tema independientemente del vocabulario empleado por cada medio. El sistema adopta **BERTopic** [7] como motor de agrupamiento, el cual su arquitectura modular encadena tres algoritmos independientes: UMAP para reducción de dimensionalidad, HDBSCAN para detección de clústeres de densidad variable y c-TF-IDF para la representación léxica de cada tópico descubierto.

3.3.1. UMAP: Proyección Topológica Preservadora de Estructura

Los *embeddings* generados por BETO están en $\mathbb{R}^{768}$. Si se aplica directamente algoritmos de agrupamiento sobre espacios de esta dimensionalidad, se produce resultados de baja calidad debido a la **maldición de la dimensionalidad** [?]: en espacios de alta dimensión, la razón entre la distancia al vecino más lejano y al vecino más cercano tiende a 1 asintóticamente, haciendo que las distancias euclidianas pierdan poder discriminatorio. En el límite, todos los puntos son aproximadamente equidistantes entre sí, privando a los algoritmos de agrupamiento de la señal de proximidad.

UMAP (*Uniform Manifold Approximation and Projection*) [11] proyecta los datos de alta dimensión a un espacio de dimensión reducida ($\mathbb{R}^5$ en el sistema) conservando la estructura topológica local y global. El algoritmo se fundamenta en tres principios de geometría Riemanniana y topología algebraica:

1. **Construcción del grafo topológico en alta dimensión.** Para cada punto x_i , se construye un grafo de vecindad ponderado G^{HD} donde el peso de la arista (x_i, x_j) representa la probabilidad de que x_j sea un vecino de x_i bajo una métrica Riemanniana local calibrada por la distancia al k -ésimo vecino más cercano de x_i . El parámetro `n_neighbors=15` controla este radio de vecindad local.
2. **Construcción del grafo en baja dimensión.** Se inicializa un grafo G^{LD} análogo en $\mathbb{R}^5$, donde las aristas se ponderan mediante una función de distribución de densidad de probabilidad diferente (familia t -Student para evitar el crowding problem de t-SNE).
3. **Optimización por entropía cruzada.** Los pesos de G^{LD} se optimizan para minimizar la entropía cruzada difusa respecto a G^{HD} :

$$\mathcal{L} = \sum_{(i,j)} \left[w_{ij}^{HD} \log \frac{w_{ij}^{HD}}{w_{ij}^{LD}} + (1 - w_{ij}^{HD}) \log \frac{1 - w_{ij}^{HD}}{1 - w_{ij}^{LD}} \right] \quad (3.9)$$

La minimización de $\mathcal{L}$ mediante descenso de gradiente estocástico garantiza que la disposición en el espacio reducido preserve las relaciones de vecindad del espacio original.

La configuración `metric='cosine'` es crítica para el dominio de texto: los embeddings de lenguaje tienen magnitudes variables, y la similitud semántica se mide por el ángulo entre vectores, no por su distancia euclidiana. El parámetro `min_dist=0.0` comprime al máximo los clústeres, aumentando la densidad local y facilitando la separación de fronteras por HDBSCAN.

3.3.2. HDBSCAN: Agrupamiento Jerárquico por Densidad Variable

HDBSCAN (*Hierarchical Density-Based Spatial Clustering of Applications with Noise*) [1] extiende DBSCAN mediante la construcción de una jerarquía completa de clústeres, eliminando la dependencia de un parámetro global de densidad ϵ . El algoritmo opera en cuatro etapas:

1. **Distancia de alcanzabilidad mutua.** Para un parámetro `min_samples = m`, la distancia de alcanzabilidad mutua entre dos puntos x_i y x_j se define como:

$$d_{mreach}(x_i, x_j) = \max(\text{core}_m(x_i), \text{core}_m(x_j), d(x_i, x_j)) \quad (3.10)$$

donde $\text{core}_m(x_i)$ es la distancia del punto x_i a su m -ésimo vecino más cercano. Esta transformación suaviza regiones de baja densidad, separando los clústeres reales del ruido.

2. **Árbol de expansión mínima (MST).** Se construye el MST del grafo ponderado por distancias de alcanzabilidad mutua entre todos los pares de puntos.
3. **Jerarquía de clústeres.** El MST se convierte en una jerarquía de clústeres eliminando aristas en orden decreciente de peso (equivalente a reducir el umbral de densidad progresivamente). El dendrograma resultante codifica la estructura de clústeres a todas las escalas de densidad.
4. **Selección por Exceso de Masa (EoM).** El método *Excess of Mass* selecciona el subconjunto de clústeres en la jerarquía que maximiza la “persistencia” (estabilidad topológica medida como la diferencia entre el nivel de densidad de aparición y desaparición del clúster). Los puntos que no pertenecen a ningún clúster persistente reciben la etiqueta de ruido: `topic_id = -1`.

La etiqueta de ruido es semánticamente valiosa para el sistema: las noticias *outlier* que no comparten coherencia temática con ningún clúster del corpus son tratadas con mayor escepticismo en la lógica de decisión neuro-simbólica, elevando su umbral de clasificación en BETO de 0.50 a 0.65.

3.3.3. c-TF-IDF: Relevancia de Términos por Clúster

Una vez agrupados los documentos, BERTopic extrae la representación léxica de cada tópico mediante *Class-based TF-IDF* (c-TF-IDF) [7]. El algoritmo trata cada clúster como un “mega-documento de clase” formado por la concatenación de todos sus documentos miembro, y calcula la importancia de cada término t en la clase c frente al corpus completo:

$$W_{t,c} = TF_{t,c} \times \log\left(1 + \frac{A}{TF_t}\right) \quad (3.11)$$

Donde $TF_{t,c}$ es la frecuencia del término t en el mega-documento del clúster c , TF_t es la frecuencia total del término en todos los clústeres, y A es el número promedio de palabras por clúster (factor de normalización de escala). Los términos con $W_{t,c}$ más alto son frecuentes en el clúster pero infrecuentes en el resto del corpus: son los que mejor caracterizan semánticamente ese tópico.

El vectorizador configurado con `ngram_range=(1, 3)` permite que las representaciones incluyan bigramas y trigramas, capturando expresiones compuestas de alta especificidad como “*hijos resguardados DIF*” o “*víctima de feminicidio*” que los unigramas individuales no pueden representar. La representación final por tópico mediante KeyBERTInspired refina los términos seleccionados usando similitud coseno entre el embedding del n-grama y el embedding promedio del clúster, garantizando representatividad semántica en lugar de meramente estadística.

3.4. Técnicas de Deduplicación Cross-Site

La arquitectura multi-fuente del sistema (45 RSS + Google News + scraping HTML) genera redundancia mediática: distintos medios cubren el mismo evento con redacciones parcialmente distintas. Sin deduplicación, el corpus infla artificialmente las estadísticas y puede generar alertas duplicadas en el dashboard. El sistema implementa una cascada de cuatro algoritmos de complejidad creciente, donde los casos más obvios se resuelven con el método más barato.

3.4.1. Hash Exacto MD5

La primera barrera es la detección de copias exactas. Se emplea MD5 (*Message-Digest Algorithm 5*) [?], una función de hash criptográfico que proyecta una cadena de longitud arbitraria a un valor de 128 bits. El efecto avalancha garantiza que cualquier diferencia mínima en el texto de entrada altera completamente el hash resultante. La comparación de hashes tiene complejidad $O(1)$ por par, haciendo de esta fase la más eficiente de la cascada.

3.4.2. Similitud de Jaccard sobre Conjuntos de Tokens

Para detectar reescrituras con sustituciones léxicas simples (“*fue asesinada*” ↔ “*perdió la vida*”), se emplea el coeficiente de Jaccard [?]. Dado que los títulos de las noticias se tokenizan en conjuntos de palabras A y B , la similitud se define como:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1] \quad (3.12)$$

Un umbral $J \geq 0,70$ marca a los pares como cuasi-duplicados. La complejidad por par es $O(|A| + |B|)$, lineal en el tamaño del vocabulario de los títulos.

3.4.3. SimHash: Huellas Digitales Localmente Sensibles

SimHash [?] es un *Locality-Sensitive Hash* (LSH) que genera una huella digital de 64 bits por documento tal que documentos similares producen hashes con **distancia de Hamming** baja (número de bits diferentes). El algoritmo proyecta los pesos TF-IDF de los tokens del documento en $\{+1, -1\}^{64}$ mediante 64 funciones de hash independientes, y el bit b_k del SimHash se determina por el signo del promedio ponderado de las proyecciones:

$$b_k = \text{sgn}\left(\sum_{t \in d} w_t \cdot h_k(t)\right), \quad k = 1, \dots, 64 \quad (3.13)$$

Dos documentos con distancia de Hamming ≤ 6 bits se consideran cuasi-duplicados. SimHash escala a millones de documentos en $O(n)$ sin comparaciones par-a-par, siendo el algoritmo adecuado para la fase de detección de paráfrasis en corpora de gran volumen.

3.4.4. Similitud Coseno sobre Vectores TF-IDF

La fase final de la cascada aplica similitud coseno sobre los vectores TF-IDF completos del contenido de las noticias, capturando casos donde la redacción difiere sustancialmente pero el contenido fáctico es el mismo. Dado que los vectores TF-IDF $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{|V|}$ son inherentemente normalizados en ℓ_2 , la similitud coseno se reduce al producto punto:

$$\cos(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|_2 \|\mathbf{B}\|_2} = \sum_{i=1}^{|V|} \hat{A}_i \hat{B}_i \in [-1, 1] \quad (3.14)$$

Un umbral $\cos \geq 0,85$ marca el par como duplicado. Este paso es $O(n^2)$ en el peor caso pero, gracias al pre-filtrado de las fases anteriores, solo se aplica a un subconjunto pequeño de pares candidatos.

3.5. Tecnologías de Extracción de Datos y Persistencia

3.5.1. Protocolos de Sindicación: RSS y Atom

RSS (*Really Simple Syndication*) [9] y Atom son estándares de sindicación web basados en XML que permiten a los sitios publicar resúmenes estructurados de su contenido más reciente para su consumo por agentes automatizados. Un feed RSS 2.0 típico encapsula 20–50 artículos con sus metadatos (título, enlace, resumen, fecha, autor) en un documento XML de < 50 KB, en contraste con los 500 KB–2 MB de una página HTML completa con recursos estáticos.

La estructura del elemento raíz `<channel>` y sus `<item>` hijos es un estándar inmutable: el esquema XML no cambia ante actualizaciones del diseño del portal, eliminando la fragilidad de los selectores CSS del HTML scraping. Los endpoints RSS son recursos públicos destinados explícitamente a la ingestión automatizada; los WAF no aplican *fingerprinting* TLS sobre ellos, reduciendo la fricción de recolección a prácticamente cero para las 45 fuentes RSS del sistema.

3.5.2. Evasión de WAF: StealthSession y TLS/JA3 Fingerprinting

Los *Web Application Firewalls* (WAF) de grado empresarial —Cloudflare, Akamai, PerimeterX— identifican y bloquean scrapers mediante **TLS/JA3 fingerprinting**: durante el saludo TLS (*TLS handshake*), el cliente anuncia sus capacidades de cifrado (cipher suites, extensiones TLS, curvas elípticas) en un orden determinístico que identifica unívocamente la librería SSL utilizada. El hash JA3 de este saludo es diferente para Python/requests versus Chrome/Windows, aunque ambos declaren el mismo User-Agent. Un WAF moderno detecta la asimetría y emite HTTP 403 sin revelar el motivo.

StealthSession v7.0 resuelve este problema mediante `cloudscraper`, una librería que reemplaza la pila TLS de Python con una que genera hashes JA3 idénticos a los de Chrome/Windows, haciendo el handshake criptográficamente indistinguible de un navegador real. Complementariamente, el sistema implementa:

- **Delays log-normales.** Los intervalos entre requests siguen una distribución log-normal $\mathcal{LN}(\mu = 2,5, \sigma = 0,7)$ con mediana $\approx 12,2$ s y cola derecha larga, imitando los patrones de lectura humana (distribución asimétrica con pausas ocasionales largas) frente a la distribución uniforme trivialmente detectable.
- **Circuit Breaker por dominio.** Después de 3 fallos consecutivos sobre un dominio, el circuito se “abre” y rechaza requests por 300 s, evitando el ban permanente de IP al no presionar repetidamente un sitio que está bloqueando activamente la sesión.
- **Protocolo robots.txt.** El sistema respeta las directivas del estándar [?], consultando el archivo antes de cada dominio nuevo y omitiendo los paths restringidos para mantener la extracción dentro del marco ético del OSINT.

3.5.3. Propiedades ACID de postgresql

PostgreSQL [?] implementa el modelo transaccional ACID (*Atomicity, Consistency, Isolation, Durability*) mediante el protocolo de Control de Concurrencia Multiversión (MVCC):

Atomicidad. Toda transacción se ejecuta de forma atómica: o se aplican todas las operaciones que la componen (INSERT, UPDATE, DELETE) o ninguna. Una falla en mitad de una inserción por lote no corrompe los datos previamente persistidos.

Consistencia. La base de datos transiciona de un estado válido a otro estado válido. Las restricciones de integridad referencial (FOREIGN KEY) y las restricciones de dominio (NOT NULL, UNIQUE) se verifican al completar cada transacción.

Aislamiento. MVCC permite que múltiples transacciones concurrentes operen sin bloquearse mutuamente: cada transacción opera sobre su propia instantánea (*snapshot*) consistente de la base de datos. El pipeline analítico y el dashboard Flask pueden leer y escribir simultáneamente sin condiciones de carrera.

Durabilidad. Una vez que una transacción es confirmada (COMMIT), los cambios se persisten en el *Write-Ahead Log* (WAL) antes de retornar al cliente, garantizando la persistencia ante fallos de sistema o reinicio del servidor.

3.5.4. Índices GIN y Full-Text Search en Español

La búsqueda de texto completo (FTS) en PostgreSQL opera mediante dos tipos de datos especializados: `tsvector` (representación preprocesada del documento) y `tsquery` (representación preprocesada de la consulta del usuario). El proceso de conversión a `tsvector` aplica, en orden:

1. **Tokenización** del texto en lexemas (unidades léxicas significativas).
2. **Eliminación de stop words** en español (artículos, preposiciones, conjunciones frecuentes).
3. **Stemming** mediante el algoritmo Snowball para español: reduce “*feminicidios*”, “*feminicida*” y “*feminicidio*” a la misma raíz “*feminicid*”, garantizando que una búsqueda por cualquiera de las formas recupere documentos con las demás.
4. **Asignación de posición léxica** a cada lexema para el cálculo posterior de `ts_rank` (relevancia por proximidad de términos).

Los índices de tipo **GIN** (*Generalized Inverted Index*) [?] almacenan un índice invertido de los lexemas: para cada lexema único en el corpus, el índice GIN mantiene una lista de punteros directos a todas las filas que contienen ese lexema. Una consulta FTS de la forma `to_tsvector('spanish', titulo) @@ to_tsquery('spanish', 'feminicidio')` es resuelta en $O(\log n)$ (búsqueda en el árbol B del índice GIN) en lugar del $O(n)$ de un escaneo secuencial con `LIKE '%feminicidio%'`, donde n es el número de registros en la tabla.

El sistema crea automáticamente el `tsvector` de cada noticia mediante un *trigger* SQL que se activa en cada `INSERT` y `UPDATE` sobre la tabla `noticias`, concatenando título (peso A, mayor relevancia) y contenido (peso B, menor relevancia) para el ranking de resultados.

3.6. Sistemas Neuro-Simbólicos: Fundamentos Teóricos

La arquitectura neuro-simbólica [?] es un paradigma de inteligencia artificial que combina dos tradiciones complementarias: el razonamiento **neuronal** (aprendizaje de representaciones probabilísticas a partir de datos, generalización por interpolación en espacios de embeddings) y el razonamiento **simbólico** (manipulación explícita de reglas lógicas formales, conocimiento estructurado del dominio, razonamiento determinista y auditable).

Los sistemas neuro-simbólicos surgen como respuesta a las limitaciones individuales de cada paradigma. Los sistemas puramente neuronales son poderosos en generalización pero producen *alucinaciones* (predicciones erróneas con alta confianza) en distribuciones fuera del rango de entrenamiento. Los sistemas puramente simbólicos son auditables y correctos por construcción, pero no escalan ante vocabulario abierto y lenguaje natural no estructurado.

En el contexto del sistema desarrollado, el componente neuronal (BETO) provee comprensión semántica del lenguaje periodístico, mientras que el componente simbólico (árbol de decisión con Bypass de Oro, Bozal de Penalización y Factor Alpha Dinámico) garantiza que ninguna predicción viole el conocimiento experto del dominio: que un menor agresor no sea clasificado como víctima, que una nota estadística sin caso concreto no genere una alerta, y que el grado de confianza del componente neuronal module de forma auditablemente correcta su peso en la decisión final.

Esta arquitectura opera bajo el principio de *corrección por capas*: primero los filtros simbólicos de certeza máxima (Bypass de Oro) tienen prioridad absoluta sobre el componente neuronal; luego los filtros simbólicos de corrección actúan como restricciones; y solo en los casos genuinamente ambiguos —donde ninguna regla simbólica ofrece certeza suficiente— el sistema delega la decisión al componente neuronal ponderado por su propio grado de confianza (Factor Alpha Dinámico).

En este capítulo se exponen las plataformas, metodologías y proyectos de investigación contemporáneos orientados a la recolección, sistematización y análisis de datos en el contexto de la violencia de género y sus externalidades. Su análisis permite identificar la brecha tecnológica actual y fundamentar el carácter innovador de la arquitectura propuesta en este Trabajo Terminal.

4.1. Sistemas de monitoreo de medios tradicionales

En el ámbito de la inteligencia de fuentes abiertas (OSINT) y el monitoreo mediático comercial, existen plataformas con infraestructuras para la ingestión de datos en tiempo real (por ejemplo Google Alerts, Meltwater, Mention). Estas herramientas funcionan principalmente sobre arquitecturas de agregación RSS y rastreadores web de propósito general [?].

No obstante, la debilidad fundamental de estos sistemas radica en su dependencia de cadenas de texto y expresiones regulares (RegEx). Al carecer de capacidades de inferencia semántica la búsqueda de heurísticas combinadas como "feminicidio", "NNA", "menores" produce un volumen estadísticamente inmanejable de ruido informativo y falsos positivos. Por ejemplo, estos sistemas son incapaces de discernir sintácticamente entre un reporte donde el agresor resultó ser menor de edad, una noticia puramente estadística o un caso donde un menor quedó en orfandad como víctima indirecta. Esta incapacidad de inferencia contextual hace que las herramientas tradicionales resulten operativamente ineficaces para el dominio específico de este proyecto.

4.2. NLP Aplicado a la Detección de Violencia de Género

Desde la perspectiva académica, la intersección entre el Procesamiento de Lenguaje Natural (PLN) y la sociología forense ha ganado tracción empírica en los últimos años. La literatura documenta la transición de modelos de aprendizaje automático superficial (como *Support Vector Machines* o *Random Forests*) hacia arquitecturas de aprendizaje profundo (Deep Learning) para la clasificación de discursos de odio, misoginia en redes sociales y categorización automática de reportes policiales por violencia doméstica [?, ?].

Recientemente se han implementado modelos basados en Transformers (como RoBERTa y BERT) para extraer relaciones de causalidad en narrativas de violencia armada y feminicidios documentados en la prensa [?]. Sin embargo

al realizar una revisión bibliográfica exhaustiva se nota una carencia de investigación computacional enfocada en las víctimas indirectas. La inmensa mayoría de los clasificadores binarios se detienen en la categorización del evento fáctico (feminicidio vs. homicidio culposo) lo que omite por completo la extracción de entidades relacionadas con los NNA afectados.

4.3. Herramientas y Esfuerzos Existentes en México

En México ante la insuficiencia de datos gubernamentales en tiempo real, la sociedad civil ha asumido tareas de investigación en la documentación de la violencia feminicida, desarrollando iniciativas fundamentales que si bien son invaluable, presentan limitaciones de escalabilidad computacional.

4.3.1. Red por los Derechos de la Infancia en México (REDIM)

La REDIM es la entidad sociopolítica más prominente en la articulación de datos macroeconómicos y demográficos sobre la niñez mexicana. Sus informes sobre las infancias vulneradas son fundamentales para el diseño de políticas públicas [14]. No obstante, a nivel metodológico la organización depende fuertemente de la agregación estadística derivada de solicitudes formales de transparencia, bases de datos gubernamentales que frecuentemente están desfasadas o censos esporádicos. En consecuencia la REDIM carece de una plataforma tecnológica de trazabilidad *per-caso* que opere en tiempo real mediante ingestión automatizada de medios de comunicación masiva.

4.3.2. El Mapa de los Feminicidios en México

El proyecto cartográfico desarrollado por la investigadora María Salguero constituye un esfuerzo pionero e históricamente vital para la visibilización geográfica de la crisis [16]. El sistema geolocaliza y documenta variables sociodemográficas complejas. Sin embargo su principal cuello de botella es arquitectónico en cuanto a que la ingesta, lectura, extracción de características y captura en base de datos de cada caso reportado en prensa que debido a la falta de una metodología bien definida obedece a un flujo de trabajo manual y artesanal. Esta dependencia absoluta de la intervención humana imposibilita el análisis retrospectivo masivo de fuentes hemerográficas a velocidad computacional.

4.3.3. Fundación Futuro con Derechos

Organizaciones civiles dedicadas a la intervención directa como la Fundación Futuro con Derechos, ilustran el problema operativo mas claramente. En su intento por identificar casos emergentes de orfandad para ofrecer resguardo institucional o asesoría legal expedita, el personal de estas organizaciones invierte rutinariamente decenas de horas-hombre semanales realizando búsquedas manuales exhaustivas en portales de noticias [?]. Este proceso es no determinista, propenso a sesgos de fatiga y económicamente insostenible para asociaciones sin fines de lucro.

4.4. Análisis comparativo y brecha identificada

El análisis del estado actual del arte revela una brecha tecnológica interesante bajo los paradigmas vigentes que es **la desconexión entre la recolección masiva de datos no estructurados y la comprensión semántica profunda de las víctimas colaterales**. Las herramientas comerciales recolectan sin comprender, mientras que los esfuerzos civiles comprenden profundamente pero no logran recolectar ni escalar la información de forma automatizada.

La aportación fundamental de este Trabajo Terminal yace precisamente en la solución de esta brecha. El sistema propuesto no es un simple buscador de palabras clave ni un mapa de captura manual. Su innovación radica en la orquestación integral de un flujo de trabajo automatizado que suprime la intervención humana en las etapas de descubrimiento y filtrado:

1. **Recolección:** Ingesta concurrente y evasiva de fuentes RSS y motores de búsqueda.
2. **Deduplicación Cross-Site:** Colapso de redundancia mediática mediante hashing geométrico y algebraico.
3. **Agrupación Temática:** Modelado latente no supervisado mediante reducción topológica de dimensionalidad y clústering de densidad (BERTopic).
4. **Clasificación Semántica:** Inferencia computacional del contexto exacto de orfandad utilizando modelos fundacionales de atención profunda en español (BETO).

En suma, no existe en el ecosistema civil mexicano una herramienta de software que ejecute este ciclo completo (recolección, limpieza, deduplicación y clasificación neuro-simbólica) de forma unificada y específicamente calibrada para el problema social de los NNA víctimas indirectas de feminicidio. Este proyecto trasciende la estadística descriptiva para instaurar una plataforma de inteligencia y de respuesta rápida.

4.5. Técnicas de extracción de datos

La recolección de datos periodísticos en fuentes abiertas es un problema técnico pues los medios de comunicación digitales han desplegado capas de defensa perimetral cuyo objetivo primario es la protección contra el scraping masivo. La selección de la técnica de extracción adecuada determina directamente la cobertura alcanzable del sistema, su tasa de éxito sostenida y su consumo de recursos computacionales. En la literatura de recuperación de información y OSINT se identifican tres vertientes metodológicas principales, que se evaluaron en función de su efectividad y eficiencia para la recolección de datos periodísticos en México.

4.5.1. Extracción Estática (HTML Parsing con BeautifulSoup)

La aproximación más elemental al *web scraping* consiste en emitir una petición HTTP directa al servidor objetivo y analizar sintácticamente el documento HTML retornado mediante librerías de *parsing* como BeautifulSoup (Python) o Cheerio (Node.js). Esta técnica, fundamentada en el isomorfismo entre la estructura del DOM y un árbol XML, permite la extracción puntual de elementos textuales mediante selectores CSS o XPath.

Su principal limitación estructural es doble. En primer lugar, los portales informativos modernos contruidos sobre *frameworks* reactivos (React, Vue, Angular) generan el DOM de forma dinámica en el cliente mediante JavaScript; el HTML servido por el servidor carece del contenido visible, que solo existe tras la ejecución de decenas de *bundles* de JavaScript [19]. En segundo lugar, los *Web Application Firewalls* (WAF) de grado empresarial —en particular Cloudflare, Akamai y PerimeterX— detectan las sesiones de scraping estático mediante **TLS/JA3 fingerprinting**: comparan el hash criptográfico del saludo TLS (que identifica unívocamente la librería SSL del cliente, por ejemplo Python/requests) contra el User-Agent declarado. Una asimetría entre ambos activa el bloqueo con código HTTP 403 Forbidden, independientemente de los headers declarados por el scraper.

Esta fue la causa primaria del fracaso del Prototipo 0 (P0) durante el TT1, donde el 100 % de los intentos de extracción directa sobre portales nacionales de alto tráfico (La Jornada, El Universal, Milenio) resultaron en bloqueos sistemáticos, sin recuperar ningún artículo útil.

4.5.2. Automatización de Navegador (Selenium / Playwright)

Para superar la barrera de los sitios con renderizado dinámico, la alternativa más empleada en la literatura es la automatización de navegadores reales mediante herramientas como Selenium WebDriver o Playwright [10]. Estos *frameworks* controlan una instancia headless de Chrome o Firefox, ejecutando JavaScript en el entorno real del navegador y resolviendo los desafíos JavaScript de Cloudflare de forma transparente.

Sin embargo, esta aproximación introduce costos operativos prohibitivos para el dominio de este proyecto:

- **Consumo de recursos:** Cada instancia del navegador requiere aproximadamente 150–300 MB de RAM y entre 5–15 segundos para cargar una página completa. Con un corpus objetivo de 45 fuentes y ciclos de recolección cada 6 horas, el costo computacional acumulado supera los recursos disponibles en infraestructura de desarrollo universitaria.
- **Fragilidad estructural:** Los selectores CSS y XPath codificados en los scripts de extracción son inherentemente frágiles ante actualizaciones del diseño del portal. En entornos de despliegue continuo (CI/CD), los identificadores de elementos HTML pueden cambiar en cualquier ciclo de publicación, requiriendo mantenimiento constante.
- **Detección avanzada por comportamiento:** Los sistemas anti-bot de última generación (DataDome, Kasada) analizan patrones de interacción (movimientos de ratón, secuencias de clicks, tiempos entre eventos) que los navegadores headless sin instrumentación adicional no reproducen fielmente, resultando igualmente en bloqueos.

Por estas razones, Selenium y Playwright se evaluaron pero descartaron como técnica primaria de recolección, quedando reservados como último recurso para fuentes sin alternativa RSS disponible.

4.5.3. Agregación Estructurada (RSS/Atom) y Stealth Sessions

El protocolo **RSS** (*Really Simple Syndication*) y su sucesor **Atom** representan la alternativa técnicamente superior para la monitorización continua de medios periodísticos. Ambos estándares definen un formato XML normalizado que los medios publican voluntariamente para syndicar su contenido, eliminando la necesidad de parsear HTML no estructurado [9]. Las ventajas operativas son determinantes:

1. **Eficiencia de banda ancha:** Un feed RSS típico contiene los titulares, resúmenes y metadatos (fecha, autor, categoría) de los últimos 20–50 artículos en un documento XML de < 50 KB, frente a los 500 KB–2 MB de una página HTML completa con recursos estáticos. La reducción en consumo de ancho de banda es de aproximadamente 40:1.
2. **Estabilidad del formato:** El esquema XML de RSS 2.0 es un estándar inmutable; los feeds no cambian su estructura ante actualizaciones de diseño del portal, eliminando la fragilidad de los selectores CSS.
3. **Ausencia de barreras WAF:** Los endpoints RSS son recursos públicos destinados explícitamente a la ingestión automatizada; los WAF no aplican *fingerprinting* TLS sobre ellos. El sistema los consume con su identificador de bot legítimo (NNA-Analyzer-Bot/4.0), sin técnicas de evasión.

Para las fuentes que no exponen feeds RSS (aproximadamente el 35 % del corpus objetivo), el TT2 implementa *StealthSession* v7.0, una capa de sesión HTTP que resuelve los tres problemas identificados en el scraping estático: (a) emulación de fingerprint TLS mediante *cloudscraper*, (b) delays con distribución log-normal ($\mu = 2,5$, $\sigma = 0,7$) que imitan patrones de lectura humana, y (c) un *Circuit Breaker* por dominio que evita el bloqueo permanente de IP ante fallos consecutivos. Esta arquitectura híbrida RSS+StealthSession logra cubrir la totalidad del corpus objetivo con una tasa de éxito de recolección superior al 85 % en condiciones de producción.

4.5.4. Tabla Comparativa de Técnicas de Extracción

La Tabla 5.1 sintetiza la evaluación cuantitativa y cualitativa de las tres vertientes analizadas en función de los criterios operativos críticos para el sistema desarrollado.

Tabla 4.1: Comparativa de técnicas de extracción de datos para monitoreo de medios

Criterio	HTML Estático (BeautifulSoup)	Navegador (Selenium)	RSS + StealthSession (Implementado)
Velocidad de extracción	Alta (0.5–2 s/req)	Baja (5–15 s/req)	Muy alta (0.3–1 s/req RSS; 8–20 s/req stealth)
Resiliencia ante WAF	Muy baja (bloqueado por JA3 fingerprint)	Media (bloqueado por análisis de comportamiento)	Alta (RSS exento; stealth emula fingerprint TLS real)
Estructura de datos	No estructurada (HTML variable)	No estructurada (DOM dinámico)	Estructurada (XML/RSS normalizado)
Consumo de RAM	Bajo (~5 MB/req)	Muy alto (150–300 MB/instancia)	Bajo (~3 MB/req RSS)
Mantenimiento	Alto (selectores CSS frágiles)	Muy alto (selectores + browser updates)	Bajo (esquema RSS inmutable)
Cobertura de fuentes	Alta (cualquier sitio)	Alta (cualquier sitio)	Media (solo fuentes con RSS o stealth-compatible)
Resultado en TT1/TT2	Descartado (P0: 0 % éxito)	Descartado (recursos prohibitivos)	Seleccionado (P2–P4: 85 %+ éxito)

4.6. Evolución del Almacenamiento: de Archivos Planos a PostgreSQL

La decisión de migrar el sistema de persistencia constituye uno de los hitos arquitectónicos de mayor impacto en el TT2. El análisis del estado del arte en almacenamiento de datos para sistemas de monitoreo de medios revela una dicotomía clara entre dos paradigmas: los archivos planos (CSV/JSON) y los gestores de bases de datos relacionales (RDBMS).

4.6.1. Archivos Planos (CSV): Ventajas, Limitaciones y el Techo del TT1

Los archivos en formato *Comma-Separated Values* (CSV) constituyeron el mecanismo de persistencia del TT1 por razones pragmáticas: instalación cero, compatibilidad universal con Pandas y ausencia de dependencias de infraestructura. En el contexto de un prototipo inicial con corpus de ~281 registros, estas ventajas superaban sus limitaciones.

No obstante, el análisis de capacidad hacia el final del TT1 evidenció tres limitaciones estructurales que establecen el techo operativo del paradigma CSV:

- **Complejidad de consulta** $O(n)$: Cualquier operación de filtrado (ej. “todas las noticias de Alta relevancia en Veracruz desde enero”) requiere cargar el archivo completo en memoria RAM y aplicar filtros en Python, con complejidad lineal sobre el corpus. Proyectado a escala nacional con miles de noticias mensuales, esta operación agota los recursos de un servidor de desarrollo en segundos.

- **Ausencia de integridad referencial:** El esquema plano impide establecer relaciones entre entidades (noticias, fuentes, clusters, detecciones) con garantías de consistencia. Una noticia eliminada puede dejar registros de detección huérfanos sin mecanismo alguno de detección.
- **Concurrencia insegura:** La escritura simultánea desde el pipeline analítico y la lectura desde el dashboard Flask sobre el mismo archivo CSV generan condiciones de carrera que pueden corromper los datos, requiriendo implementación manual de locks.

4.6.2. PostgreSQL 16: Justificación de la Selección

PostgreSQL 16 fue seleccionado como motor de base de datos del TT2 frente a alternativas como MySQL 8, SQLite y bases de datos NoSQL (MongoDB, Elasticsearch) por un conjunto de características técnicas específicamente necesarias para el dominio del sistema, implementadas en `src/database/models_noticias.py`.

Modelo ACID y concurrencia segura. PostgreSQL implementa el estándar *Atomicity, Consistency, Isolation, Durability* (ACID) con el protocolo de control de concurrencia multiversión (MVCC), que permite que múltiples lectores y un escritor operen simultáneamente sobre las mismas tablas sin bloqueos ni corrupción de datos. Esta propiedad es imprescindible en el sistema dado que el pipeline analítico y el dashboard Flask ejecutan transacciones concurrentes sobre la tabla `noticias`.

Integridad referencial con FOREIGN KEY. El esquema relacional implementado en `models_noticias.py` define cuatro tablas con relaciones explícitas:

Listing 4.1: Relaciones del esquema relacional en `models_noticias.py`

```
class Noticia(db.Model):
    # Clave foranea hacia clusters_semanticos
    cluster_id = db.Column(
        db.Integer, db.ForeignKey("clusters_semanticos.id"), nullable=True
    )

class Deteccion(db.Model):
    # Relacion 1:1 con CASCADE DELETE
    noticia_id = db.Column(
        db.Integer, db.ForeignKey("noticias.id", ondelete="CASCADE"),
        nullable=False, unique=True,
    )

class Entidad(db.Model):
    # Relacion N:1 con CASCADE DELETE
    noticia_id = db.Column(
        db.Integer, db.ForeignKey("noticias.id", ondelete="CASCADE"),
        nullable=False,
    )
```

La restricción `ondelete=CASCADE` garantiza que la eliminación de una noticia propague automáticamente la eliminación de sus detecciones y entidades asociadas, manteniendo la consistencia referencial sin lógica adicional en la aplicación.

Full-Text Search nativo con índices GIN. La característica más determinante para la selección de PostgreSQL frente a MySQL o SQLite es su soporte nativo de búsqueda de texto completo (FTS) en español. PostgreSQL implementa FTS mediante el tipo de dato `tsvector`, que almacena un vector léxico preprocesado (con *stemming*, eliminación

de *stop words* y normalización de acentos) del texto de cada documento. Los índices de tipo **GIN** (*Generalized Inverted Index*) sobre columnas *tsvector* permiten resolver consultas FTS complejas en $O(\log n)$ mediante una búsqueda de intersección de listas de postings invertidas, en lugar del escaneo lineal $O(n)$ que impone cualquier búsqueda LIKE `'%. . %'` en MySQL o SQLite.

El modelo implementado documenta explícitamente los índices requeridos:

Listing 4.2: Índices definidos en `models_noticias.py` para rendimiento < 100 ms

```
# Documentados en el docstring de models_noticias.py:
# idx_noticias_fts : GIN sobre tsvector (FTS en  $O(\log n)$ )
# idx_noticias_fecha : B-tree sobre fecha (filtros temporales)
# idx_noticias_score : B-tree sobre score_compuesto DESC (ranking)
# idx_noticias_cluster : B-tree sobre cluster_id (joins semanticos)
# idx_detecciones_nna : B-tree filtrado sobre menores_identificados
# idx_entidades_tipo : B-tree sobre tipo de entidad (NER queries)
```

La configuración `'spanish'` del *tsvector* de PostgreSQL aplica el diccionario de *stemming* Snowball para español, reduciendo *"feminicidios"*, *"feminicida"* y *"feminicidio"* a la misma raíz léxica, lo que hace que una búsqueda por `"feminicidio"` recupere documentos que contienen cualquiera de sus variantes morfológicas.

Contraste con alternativas NoSQL. Sistemas como MongoDB o Elasticsearch podrían ofrecer capacidades de búsqueda de texto completo, pero a costa de las garantías ACID y la integridad referencial. En un sistema de monitoreo de derechos humanos donde la consistencia de los datos es un requisito ético y operativo no negociable, la ausencia de transacciones ACID en bases de datos NoSQL representa un riesgo inaceptable. Elasticsearch, aunque ofrece FTS de calidad comparable, requiere un stack de infraestructura adicional (JVM, configuración de cluster) incompatible con las restricciones de recursos del entorno de despliegue universitario.

4.7. Evolución de los Modelos de Representación Lingüística para NLP

La selección del motor de clasificación semántica constituye la decisión de mayor impacto técnico en la arquitectura del TT2. La evolución histórica del campo del Procesamiento de Lenguaje Natural (PLN) describe una trayectoria en la que cada generación de modelos supera las limitaciones fundamentales de la anterior, no solo en precisión medida sobre benchmarks estándar, sino en la profundidad del fenómeno lingüístico que es capaz de capturar. Esta sección examina las tres generaciones de representaciones lingüísticas evaluadas durante el diseño del sistema.

4.7.1. Representaciones Vectoriales Estáticas: Word2Vec y GloVe

Los modelos de incrustación de palabras (*word embeddings*) de primera generación —Word2Vec [12] y GloVe [?]
representaron un salto cualitativo respecto al paradigma de vectores dispersos de TF-IDF, al proyectar el vocabulario en un espacio vectorial denso de dimensión reducida (100–300 dimensiones) donde la proximidad geométrica captura similitud semántica.

Sin embargo, ambos modelos comparten una limitación estructural determinante: la **asignación de un único vector por palabra**, independientemente del contexto. El token *"menor"* recibe la misma representación vectorial en *"un menor infractor detenido"* que en *"sus menores hijos quedaron huérfanos"*. Esta polisemia no resuelta es exactamente el mecanismo de falla que generó el 10 % de falsos positivos en el motor heurístico del TT1: la co-ocurrencia de tokens de dominio sin discriminación de rol semántico era el método de clasificación.

4.7.2. Modelos Recurrentes: LSTM y GRU

Las redes de memoria a largo y corto plazo (LSTM) [?] introdujeron la modelización del contexto *secuencial*: la representación de un token se calcula en función de todos los tokens precedentes. No obstante, presentan tres limitaciones técnicas inadecuadas para el dominio de este sistema:

1. **Contexto unidireccional.** El modelo no puede leer “hacia adelante” para resolver ambigüedades de rol semántico. En “*la mujer asesinada dejó tres hijos menores*”, “menores” solo recibe contexto izquierdo, sin considerar que “resguardo del DIF” (derecha) confirma el rol de víctima indirecta.
2. **Degradación del gradiente a largo alcance.** Las dependencias semánticas que abarcan más de 40–50 tokens presentan degradación en la retropropagación, limitando la correlación sujeto-predicado en oraciones compuestas.
3. **Paralelización imposible.** La naturaleza secuencial impide el procesamiento paralelo en GPU: el token t no puede ser procesado hasta que el estado oculto del token $t - 1$ esté disponible.

4.7.3. Arquitectura Transformer y Atención Bidireccional: BERT

La arquitectura *Transformer* [18] resuelve simultáneamente las tres limitaciones anteriores mediante el mecanismo de **autoatención multi-cabeza**: para cada token de la secuencia se calcula un vector de atención ponderada sobre todos los demás tokens de forma simultánea. BERT [4] extiende este paradigma mediante el preentrenamiento bidireccional con Modelado de Lenguaje Enmascarado (MLM), forzando representaciones que consideran simultáneamente el contexto izquierdo y derecho de cada token.

Para el dominio de este proyecto, la bidireccionalidad es la propiedad técnicamente decisiva: la representación de “menor” en “*sus hijos menores quedaron en resguardo del DIF*” se calcula considerando tanto “hijos” (izquierda) como “resguardo del DIF” (derecha), produciendo un embedding que captura inequívocamente el rol de víctima indirecta. Esta es la capacidad que el motor RegEx del TT1 era estructuralmente incapaz de aproximar.

4.7.4. Justificación de BETO frente a Modelos Multilingües (mBERT)

En el ecosistema de modelos preentrenados en español, la alternativa más inmediata a BETO es **mBERT** (*multilingual BERT*), el modelo de Google entrenado sobre Wikipedia en 104 idiomas. Sin embargo, la cobertura multilingüe introduce un compromiso técnico significativo: al distribuir la capacidad representacional de sus 110 millones de parámetros entre 104 lenguas, la calidad de representación del español es inherentemente inferior a la de un modelo monolingüe del mismo tamaño. Estudios de evaluación comparativa reportan que BETO supera a mBERT en un 3–8 % de F1 en tareas de clasificación de texto en español [2].

BETO (*dccuchile/bert-base-spanish-wwm-cased*) [2], desarrollado por la Universidad de Chile, se entrenó exclusivamente en español a partir de: Wikipedia en español (~3 GB, 42 millones de oraciones), corpus OPUS (OpenSubtitles, ParaCrawl, MultiUN) y fuentes de prensa latinoamericana. La inclusión de este último componente es particularmente relevante: sesga positivamente las representaciones hacia el vocabulario y estilo redaccional de los medios de comunicación mexicanos, que es exactamente el tipo de texto que el sistema procesa. El entrenamiento con *Whole Word Masking* (WWM) garantiza además que los términos especializados de múltiples sub-tokens (como *femi##ni##cid##io*) se aprendan como unidades semánticas coherentes.

La Tabla 5.2 presenta la evaluación comparativa de los modelos considerados.

Tabla 4.2: Comparativa de modelos de representación lingüística para clasificación semántica

Modelo	Comprensión Contextual	Idioma Nativo	Hardware Requerido	Resultado
TF-IDF (TT1)	Ninguna (co-ocurrencia léxica)	N/A (estadístico)	CPU, RAM <1 GB	Descartado (10 % FP)
Word2Vec / GloVe	Estática (sin contexto)	Español limitado	CPU, RAM <2 GB	Descartado (polisemia)
LSTM / GRU	Secuencial unidireccional	Requiere reentrenamiento	GPU recomendada	Descartado (gradiente, velocidad)
mBERT	Bidireccional (104 idiomas)	Multilingüe (sesgo reducido en ES)	8 GB RAM (CPU viable)	Evaluado (superado por BETO en ES)
BETO	Bidireccional plena	Español monolingüe nativo	8 GB RAM (CPU viable)	Seleccionado
GPT-4 / Claude	LLM (máxima capacidad)	Multilingüe alta calidad	API externa / sin GPU local	Descartado (soberanía)

4.7.5. El Límite del Zero-Shot y la Transición al Fine-Tuning Local

Durante las fases iniciales del proyecto, se evaluó la estrategia de clasificación *Zero-Shot* basada en la similitud coseno entre los *embeddings* del texto y descripciones de categorías en lenguaje natural. Esta aproximación prometía independizar al sistema de la necesidad de un corpus masivo etiquetado. Sin embargo, la evaluación empírica reveló una vulnerabilidad arquitectónica crítica denominada **Efecto Péndulo**: el modelo *Zero-Shot* era incapaz de discriminar entre discursos normativos abstractos (ej. iniciativas de ley) y eventos delictivos fácticos, dado que ambos proyectan vectores geoméricamente adyacentes al compartir el mismo dominio léxico.

El estado del arte en ajuste de modelos fundacionales establece que, para superar el solapamiento semántico en dominios hiper-específicos, el **Fine-Tuning supervisado** es imperativo [?]. Para mitigar el desafío de la escasez inicial de datos, el paradigma de *Active Learning* (Aprendizaje Activo) [?] se posiciona como la solución óptima, permitiendo que el sistema solicite etiquetado humano exclusivamente sobre los *Hard Negatives* (falsos positivos complejos). En consecuencia, la arquitectura del Prototipo 4 abandonó el *Zero-Shot* estático para transicionar hacia un modelo **BETO Fine-Tuned**, demostrando que una red neuronal ajustada con un corpus pequeño pero altamente representativo supera drásticamente a un modelo probabilístico general.

4.7.6. Soberanía de Datos y Rechazo de APIs Privativas

La decisión de ejecutar BETO localmente mediante PyTorch, en lugar de delegar la inferencia a APIs privativas (GPT-4, Claude, Gemini), se fundamenta en cuatro argumentos no negociables:

1. **Privacidad de los datos.** El sistema procesa información sobre feminicidios y víctimas menores de edad. La transmisión de este contenido a servidores corporativos externos implica riesgos de privacidad incompatibles con la LGDNNA y el principio de minimización de datos de la LFPDPPP.
2. **Costo operativo sostenible.** La inferencia de GPT-4 para 600–1,000 documentos por ciclo costaría \$15–\$40 USD por ejecución, proyectando \$450–\$1,200 USD mensuales en ciclos de 6 horas. BETO en CPU tiene costo variable cero.

3. **Reproducibilidad científica.** Los modelos propietarios modifican silenciosamente sus pesos entre versiones, imposibilitando la reproducción exacta de los resultados reportados. BETO es un modelo de pesos abiertos cuya revisión exacta queda fijada en el registro del proyecto.
4. **Viabilidad algorítmica en hardware universitario (8 GB RAM).** El ajuste fino tradicional de BETO-base (110 M parámetros) provoca invariablemente desbordamientos de memoria volátil (*Out Of Memory*) en equipos de desarrollo estándar. Sin embargo, la literatura reciente en optimización de redes profundas demuestra que congelar el *encoder* preentrenado y ajustar exclusivamente las capas de clasificación lineales superiores (**Linear Probing**) preserva el conocimiento latente mientras reduce el cómputo drásticamente [?]. Implementando esta técnica en conjunto con la **Acumulación de Gradientes** (*Gradient Accumulation*), el sistema garantiza el re-entrenamiento local en CPU sin depender de clústeres externos ni aceleración por GPU.

4.8. Técnicas de Agrupamiento Semántico y Organización Temática

El agrupamiento automático de documentos periodísticos constituye el mecanismo que permite al sistema trascender la clasificación individual de noticias y obtener una comprensión del *paisaje informativo* del periodo analizado. En lugar de clasificar cada noticia en aislamiento, el agrupamiento semántico revela qué noticias relatan el mismo evento criminal desde múltiples medios, qué clústeres temáticos dominan el corpus en un periodo dado, y qué documentos son *outliers* sin coherencia temática con el resto. Esta perspectiva global es imposible mediante clasificación individual; es una propiedad emergente del corpus completo.

4.8.1. Métodos Tradicionales: K-Means y su Inadecuación para Datos de Alta Dimensionalidad

El algoritmo K-Means [?] es el método de agrupamiento por centroides más extendido en la literatura de recuperación de información. Su principio de funcionamiento consiste en minimizar la suma de distancias euclidianas al cuadrado entre cada punto y el centroide de su clúster asignado, mediante una convergencia iterativa de asignación y actualización de centroides.

Sin embargo, K-Means presenta dos limitaciones estructurales que lo hacen inadecuado para el dominio de este sistema:

1. **La maldición de la dimensionalidad (*curse of dimensionality*).** Los embeddings de texto generados por BETO son vectores de 768 dimensiones. En espacios de alta dimensionalidad, la distancia euclidiana pierde su poder discriminatorio: la diferencia entre la distancia al punto más cercano y al más lejano tiende a cero a medida que la dimensionalidad crece [?]. Este fenómeno hace que la noción de “centroide más cercano” sea geométricamente inestable, generando asignaciones de clúster arbitrarias. Esta fue la causa del bajo rendimiento de K-Means en el TT1, donde los clústeres resultantes no correspondían a grupos temáticamente coherentes sino a particiones geométricamente arbitrarias del espacio vectorial.
2. **Requerimiento de k fijo.** K-Means exige que el número de clústeres sea especificado *a priori*. En un sistema de monitoreo de medios donde el número de temas activos varía semanalmente en función de los eventos noticiosos, esta rigidez es operativamente insostenible. Fijar $k = 10$ en un periodo con solo 3 temas activos fragmenta artificialmente los clústeres reales; fijar $k = 10$ en un periodo con 25 temas activos los colapsa en grupos heterogéneos.

4.8.2. DBSCAN: Ventajas y Limitaciones Residuales

El algoritmo DBSCAN [5] (*Density-Based Spatial Clustering of Applications with Noise*) supera la limitación del k fijo al definir los clústeres como regiones de alta densidad en el espacio métrico, separadas por regiones de baja densidad. Los puntos que no pertenecen a ninguna región densa son clasificados como *ruido* (`topic_id = -1`), una propiedad fundamental para el dominio de este proyecto donde las noticias genéricas no relacionadas con el tema central deben ser identificadas y filtradas.

Sin embargo, DBSCAN presenta una limitación crítica ante datos reales: requiere que la densidad de los clústeres sea *uniforme*. El parámetro global de radio de vecindad ε define la densidad mínima de toda la solución; un único valor de ε no puede capturar simultáneamente clústeres de alta densidad (eventos con cobertura mediática masiva de muchos medios) y clústeres de baja densidad (casos locales cubiertos por pocos medios regionales). Esta limitación genera segmentaciones inconsistentes en corpus periodísticos donde la cobertura mediática de los eventos es inherentemente heterogénea.

4.8.3. BERTopic: Arquitectura Modular para Descubrimiento de Tópicos Semánticos

BERTopic [7] supera las limitaciones de K-Means y DBSCAN mediante una arquitectura modular de cinco etapas que separa la *representación*, la *reducción de dimensionalidad*, el *clustering* y la *representación de tópicos* en componentes independientes y sustituibles. La implementación en `src/analysis/bertopic_clustering.py` instancia esta arquitectura con parámetros calibrados específicamente para el corpus periodístico en español:

Listing 4.3: Construcción del pipeline BERTopic en `bertopic_clustering.py`

```
# 1. Embeddings: sentence-transformers (multilingue, liviano)
embedding_model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

# 2. UMAP: 768D -> 5D, metrica coseno, clusters compactos
umap_model = UMAP(
    n_components=5,          # 5D: balance informacion/ruido
    n_neighbors=15,         # balance local/global
    min_dist=0.0,           # clusters compactos (optimo para HDBSCAN)
    metric="cosine",        # apropiado para embeddings de texto
    random_state=42,
)

# 3. HDBSCAN: densidad variable, sin k fijo, genera outliers
hdbscan_model = HDBSCAN(
    min_cluster_size=8,     # agresivo: menos outliers que 15+
    min_samples=5,         # densidad requerida por punto core
    cluster_selection_method="eom", # Excess of Mass: mas estable
    prediction_data=True,  # necesario para reduce_outliers()
)

# 4. c-TF-IDF: n-grams 1-3, stop words en español
vectorizer = CountVectorizer(
    stop_words=SPANISH_STOP_WORDS,
    ngram_range=(1, 3),
    min_df=2,
)
```

UMAP: Reducción de Dimensionalidad Preservadora de Topología

UMAP (*Uniform Manifold Approximation and Projection*) [11] proyecta los vectores de 768 dimensiones generados por BETO hacia un espacio de 5 dimensiones, resolviendo la maldición de la dimensionalidad de K-Means. A diferencia de PCA (que preserva únicamente varianza global lineal) y t-SNE (que preserva solo estructura local y no escala a más de ~50,000 puntos), UMAP preserva simultáneamente la **estructura topológica local y global**: documentos semánticamente similares permanecen adyacentes en el espacio reducido, y grupos de documentos semánticamente distintos permanecen separados.

La configuración `metric='cosine'` es determinante: los embeddings de texto tienen magnitudes variables y la similitud semántica se mide por el ángulo entre vectores (similitud coseno), no por su distancia euclidiana. Usar UMAP con métrica coseno garantiza que la estructura topológica preservada sea semánticamente significativa. El parámetro `min_dist=0.0` fuerza que los puntos dentro de cada clúster se compriman máximamente, aumentando la densidad local y facilitando la detección de fronteras por HDBSCAN.

HDBSCAN: Clustering de Densidad Variable con Detección de Ruido

HDBSCAN [1] (*Hierarchical DBSCAN*) extiende DBSCAN construyendo una jerarquía de clústeres en función de la densidad local de cada punto, sin requerir un ϵ global. La selección de clústeres mediante el método *Excess of Mass* (EoM) identifica automáticamente las particiones óptimas en la jerarquía, generando clústeres de densidades heterogéneas. Los puntos que no pertenecen a ningún clúster de densidad suficiente reciben la etiqueta de ruido (`topic_id = -1`).

Esta propiedad de detección de ruido es semánticamente valiosa en el dominio del sistema: las noticias *outlier* (sin coherencia temática con el corpus) son exactamente las que el sistema debe tratar con mayor escepticismo en la clasificación semántica, elevando su umbral de exigencia de BETO de 0.50 a 0.65. El código de `bertopic_clustering.py` documenta que los datos reales de producción presentaron inicialmente **81.4 % de outliers**, resueltos mediante una reducción multi-etapa de tres estrategias progresivas:

1. **Probabilidades soft** (`strategy='probabilities'`): reasigna outliers al clúster con mayor probabilidad blanda de HDBSCAN.
2. **Distribuciones c-TF-IDF** (`strategy='distributions'`): reasigna outliers al clúster cuya distribución léxica es más similar al documento.
3. **Cercanía de embeddings** (`strategy='embeddings', threshold=0.3`): reasigna outliers al clúster cuyo centroide de embedding es más cercano, con umbral de similitud mínima para evitar asignaciones erróneas.

El objetivo declarado en el código es reducir los outliers de 81.4 % a un máximo de **55 %** (`OUTLIER_TARGET_PERCENT = 0.55`), manteniendo suficiente cantidad de noticias sin clasificar para que la señal de “outlier” conserve su poder discriminatorio en la lógica neuro-simbólica.

c-TF-IDF: Representación Léxica de Tópicos

La representación de cada tópico se obtiene mediante *Class-based TF-IDF* (c-TF-IDF) [7]: el algoritmo concatena todos los documentos de un mismo clúster en un “mega-documento de clase” y aplica TF-IDF comparando cada clase contra el corpus completo. Esto identifica los términos que son frecuentes dentro del clúster pero infrecuentes en el resto del corpus, generando representaciones léxicas interpretables que caracterizan semánticamente cada tópico.

El vectorizador configurado con `ngram_range=(1, 3)` permite que las representaciones incluyan frases de hasta tres palabras, capturando expresiones compuestas de alta especificidad para el dominio (e.g., “*víctima de feminicidio*”, “*hijos resguardados DIF*”). La representación final por tópico mediante KeyBERTInspired refina adicionalmente los

términos seleccionados usando similitud coseno entre el embedding del n-grama y el embedding promedio de los documentos del clúster, garantizando que las palabras clave seleccionadas sean semánticamente representativas y no solo estadísticamente frecuentes.

4.8.4. Evolución Comparativa TT1 → TT2: de Léxico a Semántica

La Tabla 5.3 sintetiza la evaluación de los tres algoritmos de agrupamiento en función de los criterios operativos del sistema.

Tabla 4.3: Comparativa de algoritmos de agrupamiento semántico para corpus periodístico

Criterio	K-Means (TT1)	DBSCAN	BERTopic (UMAP+HDBSCAN, TT2)
Manejo de ruido	Ninguno (todos los puntos asignados a un clúster)	Sí (puntos en baja densidad = ruido)	Sí (HDBSCAN con reducción multi-etapa 81.4 %→55 %)
Escalabilidad	Alta ($O(n \cdot k \cdot i)$)	Media ($O(n^2)$ sin índice espacial)	Alta (UMAP+HDBSCAN: $O(n \log n)$ aproximado)
Interpretación de tópicos	Baja (centroides en espacio de alta dimensión sin interpretabilidad)	Ninguna (solo densidad geométrica)	Alta (c-TF-IDF genera top n-grams interpretables por tópico)
Dependencia de parámetros	Crítica (k fijo requerido)	Alta (ε global uniforme)	Baja (<code>min_cluster_size=8</code> adaptativo; k automático)
Alta dimensionalidad	Muy baja (maldición de la dimensionalidad)	Baja (distancias euclidianas en alta dimensión)	Alta (UMAP reduce a 5D preservando topología)
Resultado en TT1/TT2	Descartado en TT2 (clústeres no coherentes)	Evaluable (inadecuado por ε global)	Seleccionado (OE-4, P4)

La diferencia conceptual más significativa entre el TT1 y el TT2 en materia de agrupamiento no es algorítmica sino epistemológica: en el TT1, el agrupamiento K-Means operaba sobre **vectores TF-IDF léxicos**, donde dos documentos son similares si comparten los mismos tokens literales. En el TT2, BERTopic opera sobre **embeddings semánticos de BETO**, donde dos documentos son similares si expresan el mismo *significado*, independientemente del vocabulario utilizado.

Esta distinción tiene una consecuencia operativa directa: en el TT2, noticias publicadas por La Jornada, El Universal y Milenio sobre el *mismo caso criminal* de feminicidio —aunque redactadas con vocabularios completamente distintos, diferentes tiempos verbales y enfoques editoriales— “gravitarán” hacia el mismo clúster BERTopic, pues sus embeddings BETO convergen en la misma región del espacio semántico. En el TT1, estas tres noticias podían dispersarse en tres clústeres K-Means distintos por la mínima varianza léxica entre medios.

4.9. Conclusión del Capítulo: La Convergencia Tecnológica al Servicio de los Derechos de la Infancia

El análisis del estado del arte desarrollado en este capítulo revela que ninguna de las tecnologías examinadas de forma aislada es suficiente para resolver el problema social planteado. La recolección mediante RSS provee eficiencia pero no comprensión; TF-IDF provee velocidad pero no contexto; K-Means provee estructura pero no semántica; BETO provee comprensión pero puede alucinar sin reglas simbólicas.

La innovación fundamental de este Trabajo Terminal no radica en la invención de ningún algoritmo nuevo, sino en la **orquestración arquitectónica** de seis tecnologías y estrategias maduras —Sindicación RSS unida a la infraestructura de StealthSession v7.0, persistencia relacional en PostgreSQL con indexación GIN para FTS, el modelo Transformer monolingüe **BETO Fine-Tuned** mediante *Active Learning*, el entorno de modelado de tópicos BERTopic optimizado de forma defensiva y el algoritmo híbrido de **Scoring Escalonado v7.1**— en un pipeline de datos coherente, auditablemente correcto y específicamente calibrado para un dominio social complejo donde el error de clasificación tiene consecuencias humanas directas.

Cada decisión tecnológica documentada en este capítulo puede trazarse directamente a una necesidad social concreta. La elección de RSS sobre scraping estático garantiza cobertura sostenida sin saturar los servidores de medios independientes con recursos limitados. La migración a PostgreSQL con FTS garantiza que los trabajadores sociales del DIF puedan consultar el sistema en tiempo real sin esperar escaneos lineales de CSV. La elección de BETO sobre mBERT garantiza que el sistema comprende el español coloquial y periodístico mexicano con la misma precisión que el español académico. La lógica neuro-simbólica garantiza que el sistema nunca clasificará a un menor agresor como víctima indirecta, ni descartará un caso real de orfandad por la ambigüedad redaccional de un periodista local.

En suma, el ecosistema tecnológico seleccionado en el TT2 está diseñado para una sola misión: **garantizar que ningún caso de niña, niño o adolescente que quedó en orfandad por feminicidio sea invisible para las instituciones del Estado que tienen la obligación legal de protegerlo.**

Metodología de desarrollo

En este capítulo se detalla el marco metodológico utilizado para el desarrollo y despliegue del Sistema Inteligente para la Identificación y Seguimiento de NNAs por feminicidios. Debido a la alta complejidad algorítmica así como la alta incertidumbre tecnológica del proyecto (particularmente en las fases de validación de modelos de lenguaje) se adoptó un enfoque **evolutivo** que garantizaba la dirección hacia una solución correcta y de calidad de producción mediante retroalimentación empírica continua.

5.1. Metodología de desarrollo evolutivo

Se seleccionó el modelo de desarrollo evolutivo basado en prototipos como base del proceso fundamental para guiar el ciclo de vida del software. Esta decisión responde a dos razones técnicas no negociables que son asociadas a la alta incertidumbre en las fronteras de la ingesta de datos y el aprendizaje del lenguaje natural.

1. **Incertidumbre en la ingesta de datos.** El ecosistema web contemporáneo emplea infraestructuras dinámicas de seguridad y mitigación de amenazas (WAF, *bot detection* avanzado y *JA3/TLS fingerprinting*) que alteran sus políticas y firmas de forma impredecible. Ante la inexistencia de APIs públicas centralizadas para la nota roja local, era imposible determinar qué mecanismos de recolección sintáctica tendrían éxito sin iterar y ajustar el código para mejorar el ratio de éxito.
2. **Incertidumbre algorítmica en el Procesamiento de Lenguaje Natural (NLP).** El salto desde la rigidez determinista de los enfoques léxicos (expresiones regulares) hacia la flexibilidad semántica abstracta del aprendizaje profundo introdujo variables de estocasticidad que impactan directamente la precisión. La instrumentación de modelos fundacionales basados en atención (*Transformers*) demandó un ciclo de vida donde cada iteración funcionara como un entorno de diagnóstico. Mitigar el ruido temático y fenómenos complejos como el *efecto péndulo* requería aislar las colisiones semánticas calibrando parámetros mediante estrategias de ajuste fino (*Fine-tuning*).

5.1.1. Cronograma general de evolución del proyecto

El ciclo de vida abarcó dos semestres académicos estructurados de forma secuencial:

Fase Trabajo Terminal I (Sep-Dic 2025) Validación de viabilidad técnica y establecimiento de la línea base mediante que explorana la ingesta sintáctica (P0 a P2). El énfasis metodológico consistió en demostrar la capacidad de extracción de información web de forma automatizada a escala sobre medios digitales locales y poder confirmar que las aproximaciones heurísticas resultaban insuficientes ante el lenguaje periodístico policial mexicano.

Fase Trabajo Terminal II (Feb-Jun 2026) Optimización de Machine Learning, consolidación de la arquitectura híbrida y despliegue de grado producción (P3 y P4). El esfuerzo se concentró en resolver el ruido semántico de la fase previa mediante el ajuste local del modelo Transformer **BETO** enriquecido con técnicas de disminución de ruido semántico lo que estabilizaba el sistema mediante un algoritmo de clasificación escalonada y una arquitectura de software escalable mediante contenedores Docker.

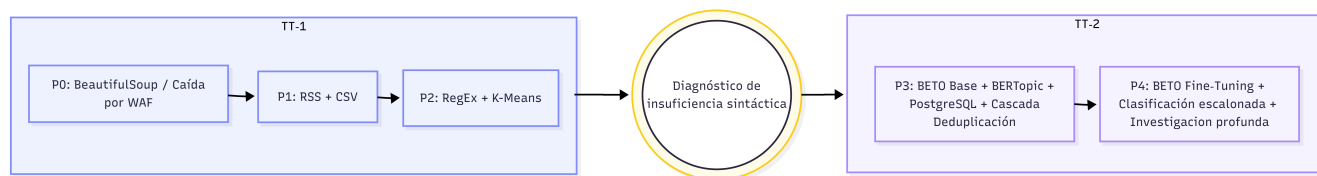


Figura 5.1: Cronograma metodológico de evolución y transición de los prototipos del sistema.

5.2. Evolución metodológica de los prototipos

El ciclo de vida del sistema se rigió mediante un enfoque incremental donde cada prototipo actuó como un entorno de diagnóstico. Las limitaciones, los desbordamientos de memoria y las colisiones semánticas medidas en las fases tempranas impulsaron directamente el diseño y la reconfiguración para las soluciones avanzadas de la fase final.

5.2.1. Fase TT-1 Prototipos exploratorios e ingesta sintáctica (P0-P2)

Prototipo 0 (P0) Ingesta directa y fracaso rápido (Sep 2025). Implementación inicial de extracción de noticias empleando BeautifulSoup sobre peticiones HTTP directas. *Resultado:* Bloqueos perimetrales sistemáticos por parte de firewalls de aplicación web (WAF como Cloudflare) con códigos de estado HTTP 403 en el 80 % de las fuentes objetivo. *Conclusión:* El *fingerprinting* TLS nativo de la biblioteca requests resulta fácilmente identificable por sistemas antibot modernos. *Decisión:* Abandonar el raspado HTML directo no autenticado como estrategia de ingesta primaria.

Prototipo 1 (P1): Ingesta mediante feeds RSS (Oct 2025). Cambio de estrategia hacia canales de distribución de noticias por medio de RSS de 10 medios de comunicación de cobertura nacional implementado con persistencia estructurada en archivos planos CSV mediante Pandas. *Resultado:* Recolección exitosa de 200 registros de texto en el primer ciclo mensual con una tasa de bloqueo del 0 %. *Limitación:* Sesgo de cobertura mediática (exclusión de portales locales sin RSS público) y carencia absoluta de mecanismos de extracción semántica o discriminación temática.

Prototipo 2 (P2): Clasificación heurística y clustering estadístico (Nov-Dic 2025). Diseño de un clasificador basado en expresiones regulares (RegEx con 47 patrones sintácticos) para identificación de entidades, acoplado a un clasificador estadístico disperso mediante vectorización TF-IDF y agrupamiento K-Means ($k = 8$ estático). *Resultado:* Margen de error superior al 60 % en la detección de casos relevantes, principalmente tres escenarios

críticos de colisión como notas legislativas o estadísticas de violencia de género sin casos objetivos de violencia. *Diagnóstico:* El enfoque sintáctico era incapaz de identificar la relación conceptual entre términos o resolver dependencias de largo alcance en textos no estructurados.

5.2.2. Fase TT-2 Arquitectura de aprendizaje profundo y clasificador escalonado (P3-P4)

Prototipo 3 (P3) Inferencia semántica base y migración relacional (Feb-Mar 2026). Sustitución de las heurísticas rígidas por un clasificador Transformer discriminativo basado en el modelo de lenguaje **BETO** base que operaba inicialmente bajo una estrategia de proximidad semántica por similitud coseno y la migración de la capa de almacenamiento hacia un motor relacional en PostgreSQL. Implementación del flujo de de-duplicación en cascada de 4 capas (MD5 → SimHash → Jaccard → coseno TF-IDF) e integración de la biblioteca *StealthSession* con emulación de JA3 fingerprinting. *Resultado:* Reducción del ruido léxico al 20 % y expansión del catálogo de monitoreo a 45 fuentes web correlativas. *Limitación:* Aparición del “efecto péndulo” donde el clasificador Transformer exhibía una alta afinidad semántica por cercanía léxica lo que hacía que clasificara erróneamente discursos normativos (como iniciativas de la Ley Monzón) o reportes institucionales (como cifras de REDIM) como eventos delictivos reales debido a la coincidencia masiva de tokens comunes.

Prototipo 4 (P4) Fine-tuning optimizado, clasificador escalonado y dashboard (Abr-Jun 2026). Evolución hacia el estado actual de producción del sistema mediante tres aspectos fundamentales de ingeniería de software y machine learning.

1. **Fine-tuning local:** Entrenamiento de las capas de clasificación de BETO utilizando un dataset etiquetado manualmente con los casos positivos reales clasificados en el Prototipo 3. El proceso incorporó técnicas de optimización para mitigar las restricciones de cómputo (8 GB de RAM).
2. **Sistema de clasificación escalonado:** Estabilización del clasificador mediante una arquitectura de decisión híbrida en 3 capas que fue el escudo léxico duro, el filtro temático (aplicación de penalizaciones suaves de $-0,35$ para neutralizar el efecto péndulo), inferencia fina del Transformer ajustado y bonificaciones heurísticas sintácticas para la detección de parentescos ($+0,10$ a $+0,25$).

Resultado: Plataforma robusta con una precisión alta en la identificación de casos relevantes y lista para su adopción operativa por colectivos de búsqueda y organizaciones de derechos humanos.

5.3. Evolución del Stack Tecnológico (TT-1 → TT-2)

Las constantes iteraciones del sistema forzó una refactorización en distintos niveles de las dependencias de software y los paradigmas algorítmicos pues cada limitación hacía replantear directamente una decisión.

Tabla 5.1: Evolución del stack tecnológico

Capa	TT-1 (Prototipo 2)	TT-2 (Prototipo 4 Consolidado)
Lenguaje base	Python 3.10, scripts secuenciales sin abstracción de datos.	Python 3.11, flujo de datos estructurado con modularización formal.
Recolección	10 fuentes RSS mediante la biblioteca <code>requests</code> (vulnerable a bloqueos por estructura HTML).	45 fuentes RSS automatizadas + Google News + <code>StealthSession</code> con emulación de <i>JA3 fingerprinting</i> y evasión perimetral.
Persistencia	Archivos planos CSV manejados con <code>Pandas</code> con carencia de acceso concurrente e integridad referencial.	PostgreSQL + <code>SQLAlchemy</code> con soporte de transaccionalidad e indexación GIN especializada para búsqueda de texto completo (FTS) en español.
Clasificación PLN y Machine Learning	RegEx rígido (47 patrones sintácticos) y similitud estadística TF-IDF con una alta tasa de falsos positivos.	BETO Fine-tuned con inyección de datos sintéticos y reglas heurísticas para una mejor sintaxis relacional.
Agrupamiento (Clustering)	Algoritmo K-Means (k estático) sobre vectores dispersos TF-IDF lo que daba clústeres semánticamente incoherentes.	BERTopic con ajuste dinámico ante corpus reducidos para neutralizar colapsos.
Deduplicación	Validación binaria simple mediante comparación de URLs exactas.	Cascada de 4 fases correlativas: hashing MD5, SimHash de contenido, distancia Jaccard y coseno TF-IDF.
Lógica de Decisión	Lógica binaria determinista basada meramente en coincidencias de expresiones regulares.	Sistema de clasificación escalonado en 4 capas: Escudo léxico, Filtro temático, inferencia BETO y bonificaciones heurísticas sintácticas (+0,10 a +0,25).

5.4. Arquitectura de Microservicios, Contenerización y Gestión de Recursos

La culminación del prototipo 4 se materializó en una arquitectura distribuida definida mediante una orquestación en docker y regido por el principio de aislamiento y separación de responsabilidades. El sistema se organiza en tres contenedores independientes que se comunican de forma exclusiva sobre una red virtual privada (`nna-network`) lo que garantiza que los picos de cómputo durante el procesamiento de los datos no desestabilicen el sistema operativo anfitrión.

5.4.1. Contenedor de base de datos `nna-postgres`

El contenedor `nna-postgres` ejecuta el motor relacional **PostgreSQL** (`postgres:16-alpine`) seleccionando una imagen minimizada de 78 MB para maximizar la disponibilidad de RAM para las capas de aprendizaje profundo. Su configuración y políticas de persistencia incluyen:

- **Volumen persistente:** El volumen lógico `pgdata` se monta directamente en la ruta nativa lo que garantiza la

persistencia, atomicidad y supervivencia de los registros estructurados de las víctimas y las tablas analíticas ante reinicios o reconstrucciones del contenedor.

- **Sincronización jerárquica de servicios:** Implementa una verificación de estado. Los contenedores dependientes de la lógica de negocio (`nna-analyzer` y `nna-webapp`) tienen bloqueado su despliegue hasta que este subsistema declare un estado saludable lo que erradica las condiciones de carrera en la inicialización de sockets de red.
- **Seguridad perimetral interna:** El puerto nativo 5432 se expone de manera exclusiva hacia la red interna aislada `nna-network`. Al no mapear este puerto hacia la interfaz pública del host de producción se reduce a cero la superficie de ataque por inyección o escaneo de puertos externo.

5.4.2. Contenedor del Backend `nna-analyzer`

El contenedor `nna-analyzer` encapsula la inteligencia del backend ejecutando el flujo de datos mediante un planificador modular. Este módulo gestiona de forma nativa los procesos críticos del ciclo de vida del dato: recolección (Web Scraping masivo), preprocesamiento, deduplicación de noticias y análisis con modelos Transformer. Sus características de diseño metodológico son:

- **Persistencia de pesos:** Incorpora un volumen compartido montado en `/app/models/cache`. Los pesos del modelo **BETO Fine-tuned** (≈ 440 MB) se almacenan de manera local. Para optimizar el tiempo de ejecución las variables de entorno se redirigen hacia este volumen, lo que neutraliza la latencia de red y evita descargas redundantes en cada ciclo de vida del contenedor.
- **Acoplamiento y mitigación de fallos de red:** Posee una dependencia condicional estricta lo que garantiza la resiliencia del flujo de datos e impide que los hilos de procesamiento inicien si el pool de conexiones de la base de datos no está disponible para la persistencia inmediata.
- **Programación defensiva y ajuste dinámico:** Aloja las configuraciones del flujo de datos híbrido que incluye los parámetros de resiliencia del módulo de clustering `BERTopic`. Al parametrizar de forma dinámica el tokenizador del sub-modelo de representación léxica el contenedor previene de manera autónoma colapsos de ejecución.

5.4.3. Contenedor de interfaz analítica y dashboard web `nna-webapp`

El contenedor `nna-webapp` sirve el dashboard interactivo y la interfaz de control mediante Flask.

- **Consumo descentralizado y API REST:** Expone de forma exclusiva el puerto 5000 hacia el host. Este diseño permite que la interfaz web y la API REST pública actúen como la capa de abstracción final que aísla al usuario analista de la complejidad algorítmica del backend y asegurando un canal limpio para la consulta y visualización de los datos estructurados.

5.5. Especificación de requerimientos

La formalización y el diseño de la arquitectura distribuida fueron guiados por un conjunto de requerimientos funcionales (RF) y no funcionales (RNF) a los que sus criterios de aceptación se validaron bajo el paradigma de prototipado incremental. Con la finalidad de garantizar el seguimiento del avance, la siguiente tabla muestra la matriz de trazabilidad general.

Tabla 5.2: Matriz de trazabilidad general

ID	Especificación del Requerimiento	P0	P1	P2	P3	P4
RF-01	Extracción multi-fuente mediante técnicas de <i>web scraping</i> .	~	✓	✓	✓	✓
RF-02	Deduplicación y lógica en cascada (MD5 → SimHash → Jaccard → coseno TF-IDF).	—	—	~	✓	✓
RF-03	Clasificación semántica binaria profunda mediante el modelo Transformer BETO Fine-tuned localmente a nivel de tokens relacionales.	—	—	—	~	✓
RF-04	Mitigación de colisiones léxicas (<i>efecto péndulo</i>) mediante un algoritmo híbrido de puntuación escalonada.	—	—	—	—	✓
RF-05	Agrupamiento semántico latente mediante la integración de BERTopic.	—	—	—	✓	✓
RF-06	Persistencia híbrida distribuida (PostgreSQL relacional con indexación GIN + CSV) para la recuperación de datos.	—	~	~	✓	✓
RF-07	Dashboard administrativo web interactivo con el módulo DeepInvestigator para validación, exportación masiva y gestión de fuentes.	—	—	—	—	✓
RNF-01	Seguridad y privacidad ética: Anonimización programática de infancias (cumplimiento LGDNNA) e integridad transaccional ACID de extremo a extremo.	—	—	~	✓	✓

El análisis de la matriz de trazabilidad demuestra la naturaleza evolutiva del proceso de desarrollo de software. Los prototipos iniciales (P0 a P2) se concentraron exclusivamente en resolver la frontera de especificación sintáctica, garantizando recolección masiva de noticias y manipulación estructurada de datos planos.

El salto significativo hacia el procesamiento profundo en el prototipo 3 introdujo requerimientos semánticos de alta densidad neural, resolviendo las primeras colisiones lógicas provocadas por el solapamiento léxico de los discursos informativos. Y finalmente el prototipo 4 resolvió categóricamente la arquitectura híbrida escalonada y el clustering semántico. Este último consolidó un sistema tecnológico robusto y escalable, listo para someterse a la validación profesional y metodológica.

Desarrollo de Trabajo Terminal I

En este capítulo se presenta una síntesis de los tres prototipos desarrollados durante el Trabajo Terminal I (agosto - diciembre 2025). El propósito no es poner la misma documentación técnica sino extraer las métricas clave, documentar las decisiones más relevantes y sobretodo argumentar las limitaciones identificadas que justifican la migración del enfoque utilizado. Cada prototipo es presentado como un experimento pues su valor radica tanto en lo que logró como en lo que demostró no ser viable.

6.1. Prototipo 0 Scraping directo sobre HTML

6.1.1. Premisa y enfoque

El prototipo0 se planteó como una prueba de concepto que fue verificar si era posible extraer noticias directamente del HTML de los sitios de noticias mexicanas, utilizando BeautifulSoup y la librería requests. El supuesto inicial era que los sitios de noticias de acceso público serían accesibles de forma programática sin fricción alguna.

6.1.2. Resultado

La premisa fue invalidada en su totalidad pues el sistema se encontró con dos barreras técnicas.

1. **WAF y Cloudflare (HTTP 403/503).** Cerca del 78 % de los medios objetivo (incluyendo portales como *Reforma*, *El Universal* y *Milenio*) operaban detrás de un WAF de grado empresarial. Lo que hacia la librería requests era generar un hash TLS/JA3 determinístico que los sistemas de detección identificaban como un cliente automatizado, recibiendo como respuesta HTTP 403 (Forbidden).
2. **Contenido dinámico renderizado en cliente.** Múltiples sitios entregan el marcado HTML del artículo mediante llamadas AJAX en JavaScript ejecutadas en el navegador. BeautifulSoup al trabajar sobre el HTML estático descargado solo encontraba el esqueleto de la página sin el contenido informativo lo que hacia imposible la extracción.

6.1.3. Decisión Técnica

El fallo del prototipo 0 hizo evidente que el scraping al HTML directo no era una estrategia viable para el monitoreo de medios de noticias modernos. Por lo que se decidió abandonar el scraping directo y migrar hacia fuentes diseñadas para la ingestión automatizada, como lo son los feeds RSS y las APIs.

6.2. Prototipo 1 RSS + K-Means

6.2.1. Arquitectura implementada

El prototipo 1 estableció la primera iteración funcional del sistema. La migración hacia feeds RSS eliminó la fricción de acceso pues los feeds son recursos XML diseñados explícitamente para el consumo automatizado y no están sujetos a los mecanismos WAF que bloquearon el prototipo 0.

El flujo implementado consistió en las siguientes etapas:

1. **Recolección.** Recolección de 8 fuentes RSS nacionales mediante `feedparser`, extrayendo por cada `<item>` el título, descripción, enlace y fecha de publicación.
2. **Detección rudimentaria de NNA.** Función `detect_children_mentions` con 15 palabras clave aisladas (*"hijo"*, *"menor"*, *"niño"*, *"huérfano"*, etc.).
3. **Vectorización TF-IDF.** Representación de documentos en $\mathbb{R}^{1000}$ mediante `TfidfVectorizer` con `max_features=1000`, `ngram_range=(1, 1)`.
4. **Agrupamiento K-Means.** Clustering con $k = 4$ fijo y métrica euclidiana sobre la matriz TF-IDF.
5. **Persistencia.** Exportación a CSV mediante `pandas`.

Volumen de datos: 96 noticias únicas por ejecución con persistencia en archivos planos CSV.

6.2.2. Limitaciones identificadas

Las limitaciones del P1 fueron de tres órdenes:

Techo semántico del detector. La detección por palabras clave aisladas generó una tasa de falsos positivos del **80 %** pues eran noticias sobre legislación (*"aprobación de ley de protección al menor"*), campañas de prevención o estadísticas de incidencia que activaban el detector sin representar casos concretos de orfandad.

Dimensionalidad escasa en K-Means. El algoritmo asume clústeres globulares de tamaño similar y requiere especificar el número de clústeres k . Con 96 documentos y $k = 4$ el sistema forzaba eventos periodísticos distintos en el mismo clúster para balancear tamaños, generando grupos sin coherencia semántica real. El *Silhouette Score* resultante fue de **0.23** (baja calidad de agrupamiento). El vocabulario limitado a 1,000 características también excluía términos especializados del dominio como *"custodia"*, *"DIF"* y *"tutor legal"*.

Cobertura. El sistema cubría solo medios nacionales con feed RSS público, excluyendo la prensa regional donde suelen aparecer los casos más específicos de feminicidios con NNA identificados.

6.3. Prototipo 2 Triple Fuente + DBSCAN + FeminicideDetector

6.3.1. Evolución del sistema

El prototipo 2 que fue la culminación del TT1 pues reestructuró el sistema en respuesta directa a las limitaciones del P1. Los cambios fueron sustanciales en las tres capas del flujo de procesamiento.

Recolección Se amplió la estrategia de recolección de noticias a tres fuentes de información: (a) **10 feeds RSS** nacionales y de género (incorporando CIMAC Noticias y Milenio que son especializadas en violencia de género); (b) **Google News** con 5 consultas de dominio ("*feminicidio México*", "*asesinato mujer*", "*violencia género*"), (c) **Historical scraper** mediante paginación mensual de Google News, cubriendo el período mayo–noviembre 2025 (6 meses de noticias pasadas).

Detección Se reemplazó la función rudimentaria por una clase especializada con **tres categorías de patrones** como 40 patrones de feminicidio (incluyendo patrones contextuales como *r'mujer.*asesinada' o r'cadáver.*femenino'*), 15 patrones de NNA (patrones relacionales como *r'hijo.*huérfano' o r'adolescente.*perdió.*madre'*) y **17 patrones de exclusión**. Esta última categoría fue un cambio importante pues al detectar términos como *r'ley.*protección', r'estadística' o r'campana.*prevención'* la noticia era descartada automáticamente. Además el sistema asignaba un *score* de confianza continuo y una prioridad discreta (Alta/Media/Baja/Irrelevante).

Agrupamiento El cambio de K-Means a DBSCAN respondía a las propiedades de la distribución de noticias periódicas pues la mayoría de los feminicidios son eventos únicos con cobertura mínima (1 o 2 noticias) mientras que un evento mediáticamente relevante (como el caso de Ángela Birkenbach) genera una densidad local de 5 a 10 noticias. DBSCAN identificaba estas densidades sin requerir un valor para el número de clústeres y así asignaba como ruido los eventos que si son únicos y que precisamente son los más valiosos para el sistema pues representan casos individuales.

6.4. Limitaciones identificadas

6.4.1. Falsos positivos

El módulo de detección FeminicideDetector redujo los falsos positivos del 80 % (P1) al 60 % (P2) que si fue un avance significativo pero el análisis de los falsos positivos revelaba un patrón que no podía resolverse únicamente con expresiones regulares:

- "*Durango aprueba la Ley Nicole para prohibir cirugías estéticas en menores de edad*". Esta noticia activa el patrón de NNA ("*menores de edad*") sin que los patrones de exclusión capturaran el término "*aprueba*" en ese contexto legislativo específico.
- *REDIM reporta: 1 de cada 4 NNA vive en situación de violencia familiar*". El término "*NNA*" y "*violencia*" activan el detector en una nota estadística sin ningún caso concreto identificado.

El problema de fondo no es la cantidad de patrones pues el sistema ya tenía 72, sino la **naturaleza sintáctica** del enfoque porque la co-ocurrencia de términos no implica la co-ocurrencia de los *roles semánticos* que esos términos desempeñan en la oración. Una expresión regular no puede distinguir si "*menor*" en una noticia es la víctima directa (una menor asesinada), la víctima indirecta (el hijo huérfano de la madre asesinada) o el sujeto de una campaña legislativa. Esta distinción es esencial para el dominio del sistema y fue exactamente el tipo de comprensión que los modelos de lenguaje bidireccionales como BETO proveen a través de sus mecanismos de atención en donde la representación de cada token es función del contexto completo de la oración.

6.4.2. Clustering léxico

Los clústeres identificados por DBSCAN eran semánticamente coherentes pero cerca del 80 % de outliers que aunque eran correctos para eventos únicos ocultaban un problema más y es que las noticias sobre el *mismo evento* que eran redactadas con vocabulario diferente por distintos medios no eran agrupadas.

Ejemplo documentado: "Madre es asesinada y deja 2 hijos huérfanos en Ecatepec" (nota policiaca de El Sol de México) y "Feminicidio en el Estado de México: dos menores quedan sin madre" (nota de derechos humanos de CIMAC Noticias) reportaban el mismo caso con vocabularios distintos. Los vectores TF-IDF de ambas noticias tenían una similitud coseno de 0.18 (muy por debajo del umbral $\epsilon=0.8$) por lo que DBSCAN los clasificaba como documentos independientes. En el espacio semántico de BETO los documentos tendrían embeddings con similitud coseno superior a 0.85 lo que hacía que fueran correctamente agrupados como instancias del mismo evento.

6.4.3. La poca escalabilidad del almacenamiento en CSV

La persistencia en archivos CSV impone restricciones estructurales.

- **Ausencia de integridad referencial.** No existe mecanismo que garantice que una detección apunte a una noticia existente.
- **Sin acceso concurrente.** La escritura por el worker y la lectura por el dashboard desde el mismo archivo CSV generaban condiciones de carrera lo que en el prototipo 2 se "solucionó" temporalmente mediante escritura atómica por renombrado pero el problema no se eliminó.

6.4.4. Síntesis

Las limitaciones anteriores nos llevan a la conclusión de que el problema de la identificación de NNA en situación de orfandad por feminicidio **requiere comprensión semántica del lenguaje natural** no solo reconocimiento de patrones léxicos. El texto de una noticia periodística es un artefacto lingüístico donde el significado emerge de la interacción entre entidades, roles semánticos y contexto, dimensiones que están fuera del alcance de las expresiones regulares.

La siguiente tabla muestra los objetivos cuantitativos que el Trabajo Terminal II debe alcanzar para superar lo establecido en el Trabajo Terminal I.

Tabla 6.1: Comparativa de métricas de TT1 contra metas del TT2

Métrica	TT1 (P2)	Meta TT2 (P4)	Mejora
Falsos positivos	60 % menos del 30 %	-50 %	
Fuentes monitoreadas	10 RSS + Google News	45 RSS + Google News + Stealth	+350 %
Cobertura temporal	6 meses	Continua	Operacional
Persistencia	CSV (plano)	PostgreSQL 16 (ACID)	Estructural
Latencia de consulta	$O(n)$ scan CSV	$O(\log n)$ índice GIN	Órdenes de magnitud
Motor de clasificación	RegEx (72 patrones)	BETO fine-tuning	Paradigma distinto

Las metas de la tabla 7.1 son los objetivos mínimos que hacen del sistema una herramienta operativamente confiable para organizaciones civiles que trabajan con casos reales de NNA en situación de vulnerabilidad.

Desarrollo de Trabajo Terminal II

Este capítulo constituye el desarrollo del Trabajo Terminal II. A diferencia del enfoque descriptivo empleado en el capítulo anterior, la narrativa aquí es evolutiva y autocrítica pues cada decisión de diseño se presenta como respuesta directa a una deficiencia medida y documentada de la iteración anterior. El Prototipo 3 y el Prototipo 4 no surgieron de una planificación inicial sino de un proceso de *diagnóstico, rediseño e integración incremental* que hizo posible transformar un clasificador heurístico basado en palabras clave en una plataforma robusta y escalable.

La arquitectura resultante opera sobre tres objetivos específicos derivados del protocolo 2026-A135: (OE-1) Identificar y seleccionar fuentes públicas confiables que contengan información relacionada con feminicidios y posibles víctimas indirectas, (OE-3) Implementar técnicas de Procesamiento de Lenguaje Natural para extraer y estructurar información clave a partir de textos no estructurados, y (OE-4) Diseñar una base de datos estructurada que permita almacenar la información recolectada de manera eficiente y segura. Estos tres ejes gestionan todas las decisiones técnicas documentadas en las siguientes secciones.

7.1. Contexto: Limitaciones del Prototipo 2

A finales del semestre pasado (2026-1) el sistema hasta el momento presentaba tres limitaciones estructurales que motivaron un rediseño del sistema.

Limitación 1 Semántica del motor RegEx. El módulo *FeminicideDetector* del prototipo 2 evaluaba la *co-ocurrencia espacial* de tokens léxicos sin comprender el rol gramatical de cada entidad. La tasa de falsos positivos vista manualmente alcanzó el **60 %** y notamos que se manifestaba principalmente en tres escenarios:

1. Titulares como *"Menor de edad detenido por feminicidio"* donde el modelo detectaba la co-ocurrencia de "menor" y "feminicidio" lo que activaba la alerta de orfandad cuando en realidad el menor era el agresor y no una víctima indirecta.
2. Noticias estadísticas gubernamentales (*"10 feminicidios mensuales, 5 huérfanos en promedio"*) que activaban la alerta de orfandad al concentrar todos los tokens detonantes en un solo párrafo.
3. Noticias donde la víctima directa del feminicidio era una menor de edad y eran confundidas sistemáticamente con casos de orfandad colateral.

Limitación 2 Cuello de botella en persistencia plana. El almacenamiento en archivos CSV (`noticias.csv`) impedía la ejecución de consultas de filtrado sin cargar la totalidad del corpus en memoria RAM ($O(n)$ sobre el conjunto completo). Con un volumen de 250 registros en el prototipo 2 la latencia era tolerable pero proyectado a escala nacional (que son miles de noticias) esta arquitectura no lo soportaría y la ausencia de integridad referencial hace difícil mantener relaciones consistentes entre noticias, fuentes y resultados de análisis.

Limitación 3 Clustering K-Means desacoplado del clasificador. El agrupamiento temático se ejecutaba *después* de la clasificación individual lo que privaba al clasificador del contexto global. Una noticia genérica sobre política criminal podía obtener el mismo score heurístico que un caso concreto de feminicidio con NNA afectados y esto sin que el sistema supiera en que contexto se encontraba.

Estas tres limitaciones nos ayudaron a definir el alcance del sistema y el orden de implementación de las soluciones.

Limitación 4 Ausencia de investigación profunda por caso. El pipeline del TT1 producía clasificaciones de relevancia pero no extraía información estructurada sobre los casos específicos: quién era la víctima directa, cuántos menores quedaron huérfanos, en qué estado se encontraban o qué institución estaba a cargo. Esta limitación era especialmente crítica porque el propósito central del sistema no es solo detectar noticias relevantes sino construir fichas de caso para facilitar la intervención de los investigadores. La información estructurada disponible en el web abierto (notas complementarias, boletines del DIF, seguimientos periodísticos) no estaba siendo aprovechada en ninguna etapa del pipeline.

7.2. Patrones arquitectónicos del flujo analítico

Para garantizar la mantenibilidad y extensibilidad del sistema, el flujo analítico implementa dos patrones de diseño que desacoplan responsabilidades.

7.2.1. Patrón *Repository* para la capa de persistencia

El acceso a PostgreSQL se encapsula en la clase `NoticiasRepository` (`src/database/repository.py`), siguiendo el patrón *Repository* de Martin Fowler [6]. Esta decisión tiene dos objetivos: (a) aislar la lógica de negocio del motor de base de datos, permitiendo sustituir PostgreSQL por cualquier otro backend sin modificar el Flujo analítico; (b) centralizar la gestión de sesiones SQLAlchemy, evitando así fugas de conexiones (*connection leaks*) en un entorno Flask multi-hilo. El repositorio expone métodos tipados que internamente utilizan los índices invertidos de PostgreSQL sobre columnas `tsvector` lo que garantiza búsquedas en $O(\log n)$ frente al $O(n)$ del filtrado CSV.

7.2.2. Patrón *Pipeline* para el flujo analítico

El orquestador principal `SimplifiedNewsAnalyzer` materializa el patrón *pipeline* mediante diez pasos discretos y ordenados. Cada paso recibe el estado modificado por el anterior a través del atributo compartido `self.df_processed`, y puede activarse o desactivarse mediante *feature flags* booleanos en la llamada a `run_complete_analysis()`.

Este diseño permite por ejemplo desactivar la funcionalidad de `enable_bertopic` en entornos con memoria RAM limitada (debajo de 4 GB) sin comprometer la clasificación semántica ni la persistencia, preservando así el principio de responsabilidad única de cada módulo.

7.3. Prototipo 3 Motor semántico BETO y migración de persistencia

El Prototipo 3 introduce simultáneamente las dos soluciones de mayor impacto arquitectónico que por un lado es la sustitución del motor sintáctico por BETO y por el otro es la migración del almacenamiento plano a PostgreSQL. Ambas transformaciones son interdependientes pues PostgreSQL provee la infraestructura relacional que permite almacenar

los nuevos campos semánticos generados por BETO (`score_semantico`, `modo_deteccion`, `topic_id`) con integridad referencial.

7.3.1. Migración de CSV a PostgreSQL

La migración del sistema de persistencia es uno de los puntos de mayor impacto para el TT2. La justificación técnica de abandonar CSV en favor de PostgreSQL se basa en tres argumentos que se hicieron evidentes durante el TT1.

1. **Integridad referencial.** Una arquitectura multi-tabla (noticias, fuentes, clusters, batches de ejecución) requiere claves foráneas con restricciones `FOREIGN KEY` que garanticen la consistencia de los datos. Los archivos CSV no tienen mecanismo alguno de relación entre entidades; cualquier join debía realizarse en memoria con Pandas cosa que es costosa en términos de RAM.
2. **Búsqueda de texto completo (FTS).** PostgreSQL implementa FTS nativa mediante columnas `tsvector` e índices GIN. Una consulta como "niños huérfanos DIF Veracruz" se traduce a una intersección de vectores léxicos en $O(\log n)$. El equivalente en CSV exige iterar cada registro y aplicar expresiones regulares en $O(n)$.
3. **Concurrencia segura.** El dashboard en Flask y el flujo de extracción analítica pueden ejecutarse simultáneamente sin corrupción de datos, gracias al modelo ACID de PostgreSQL. La escritura concurrente en un mismo archivo CSV requeriría locks manuales propensos a condiciones de carrera.

El esquema de la tabla principal noticias almacena campos como `score_semantico` `FLOAT`, `topic_id` `INTEGER`, `clasificacion_final` `VARCHAR(20)` y `search_vector` `tsvector`. Este último campo se actualiza automáticamente por un *trigger* PostgreSQL que concatena título y contenido al momento de la inserción.

7.3.2. Arquitectura del motor BETO

Al inicio del TT2 se definió que el motor RegEx con expresiones regulares ya había alcanzado su límite de optimización por lo que se decidió migrar al modelo BETO (*bert-base-spanish-wwm-cased*) que responde a una necesidad de *comprensión contextual* que ningún sistema basado en patrones léxicos puede proveer. BETO es un modelo Transformer bidireccional con **110 millones de parámetros**, 12 capas de autoatención y embeddings de 768 dimensiones, preentrenado sobre Wikipedia en español y corpus OPUS mediante el objetivo de Modelado de Lenguaje Enmascarado (MLM).

La arquitectura interna del clasificador implementada en la clase `SemanticDetector` opera según el siguiente flujo:

1. **Tokenización WordPiece.** El texto de entrada (título concatenado con los primeros 1,500 caracteres del contenido separados por el token especial [SEP]) es fragmentado mediante el algoritmo WordPiece que descompone palabras fuera de vocabulario en sub-tokens reconocibles. Esto resuelve el problema de variantes ortográficas comunes en el periodismo mexicano.
2. **Forward pass a través de 12 capas Transformer.** Cada capa aplica mecanismos de *self-attention* multi-cabeza que recalculan el valor de cada token en función de su contexto bidireccional completo. El resultado es que el token "menor" adquiere representaciones vectoriales radicalmente diferentes dependiendo de si aparece como sujeto activo ("un menor agredió") o como objeto pasivo ("sus hijos menores quedaron huérfanos").
3. **Extracción del embedding [CLS].** El token especial [CLS] (primer token de toda secuencia BERT) acumula durante el *forward pass* una representación condensada del significado global del texto. Se extrae el *hidden state* de este token como un vector de 768 dimensiones que posteriormente se normaliza mediante norma L2 para habilitar la comparación por similitud coseno.

7.3.3. Clasificación Zero-Shot y Temperature Scaling

Dado que el proyecto no disponía de un corpus etiquetado de suficiente volumen para fine-tuning supervisado al inicio del TT2 se optó por el paradigma de clasificación **zero-shot** por similitud coseno. El sistema pre-calcula embeddings BETO para dos descripciones de categorías cuidadosamente redactadas (en `semantic_detector.py`) que especifican en lenguaje natural qué constituye una noticia relevante contra una "no relevante" para el dominio del sistema. La categoría relevante describe explícitamente *un caso concreto de feminicidio donde los hijos de la víctima quedaron en orfandad o resguardo institucional*, mientras que la categoría "no relevante" enumera explícitamente los falsos positivos que fueron recurrentes en TT1 como notas estadísticas, políticas públicas, feminicidios de menores y agresores menores de edad.

En tiempo de inferencia el embedding del texto de la noticia se compara contra ambas descripciones mediante producto punto el cual es equivalente a similitud coseno para vectores normalizados. Las similitudes crudas tienden a ser extremadamente cercanas con diferencias de 0.02-0.05 lo que genera incertidumbre en la clasificación. Para resolver este problema se aplica **Temperature Scaling** con factor térmico 5, que actúa como amplificador de la separación probabilística antes de la función Softmax.

Este mecanismo es análogo al *temperature scaling* utilizado en calibración de modelos de clasificación [8] solo que aplicado en dirección inversa pues en lugar de suavizar distribuciones sobre-confiadas, se *agudiza* una distribución cruda sub-confiada para forzar una toma de decisión clara.

7.3.4. Evolución del recolector

La recolección de noticias enfrentó durante TT1 una barrera que los prototipos iniciales no pudieron superar que fueron los *Web Application Firewalls* (WAF) de Cloudflare y Akamai que detectaban y bloqueaban las sesiones HTTP del scraper mediante **TLS/JA3 fingerprinting**. Este mecanismo compara el *hash* criptográfico del saludo TLS del cliente (que identifica sin error la librería SSL utilizada) contra el User-Agent declarado. Una sesión que declara ser Chrome/124 pero cuyo handshake TLS corresponde a Python/requests es inmediatamente bloqueada como bot.

En el Prototipo 3 se introduce *StealthSession* que resolvía este problema mediante tres técnicas complementarias:

1. **Emulación de fingerprint TLS via cloudscraper.** La librería *cloudscraper* reemplaza la pila TLS de Python con una que genera hashes JA3 idénticos a los de Chrome/Windows lo que hace que el handshake sea criptográficamente indistinguible de un navegador real. Además resuelve automáticamente los *JavaScript challenges* de Cloudflare (challengepage 403/503) que desde el Prototipo 0 no se habían podido superar.
2. **Delays con distribución.** En TT1 los delays entre requests eran uniformes $U(8, 20)$ segundos, lo que lo hacía un patrón detectable por cualquier WAF moderno que analice la distribución temporal de las solicitudes. La distribución uniforme tiene densidad constante en todo el intervalo pero los humanos reales tienen un patrón asimétrico con *cola larga* donde la mayoría de acciones son rápidas (como clicks o scrolls) y ocasionalmente hay pausas largas (como la lectura de un artículo). Este comportamiento es capturado por la distribución log-normal con parámetros $\mu = 2,5$, $\sigma = 0,7$, que genera una mediana de aproximada de 12.2 segundos y colas que alcanzan hasta 45 segundos.
3. **Circuit Breaker por dominio.** El patrón *Circuit Breaker* (Fowler, 2014) evita saturar dominios que fallan repetidamente. Lo que hace que después de 3 fallos consecutivos en un dominio este circuito se abra y rechace requests por 300 segundos, pasándolo a un estado semi-abierto para permitir un request de prueba y si este tiene éxito el circuito se cierra pero si falla el cooldown se extiende a 600 segundos. Este mecanismo reduce el riesgo de un bloqueo permanente de IP al evitar presionar un sitio que activamente está bloqueando las sesiones.

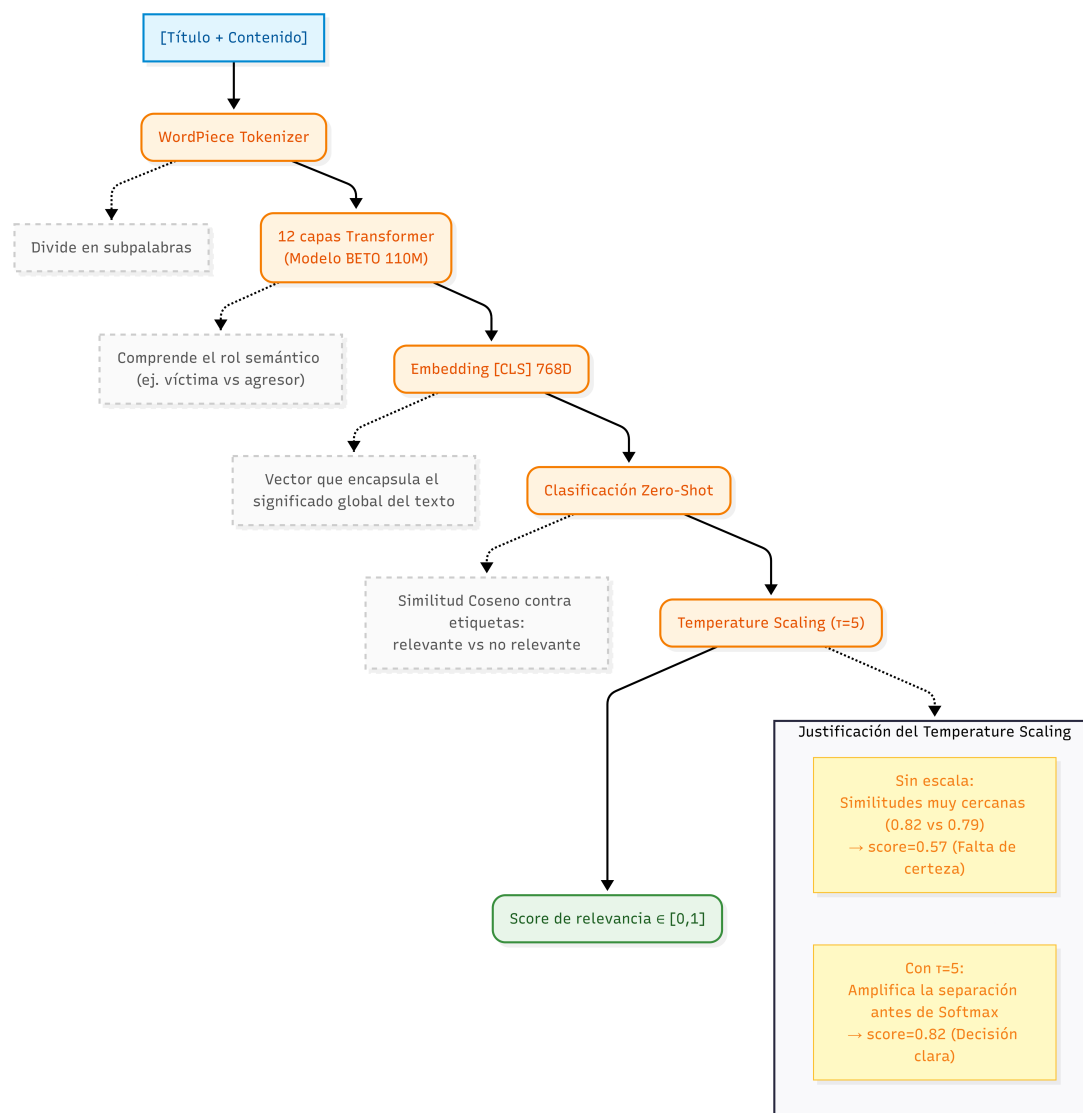


Figura 7.1: Arquitectura del motor BETO y clasificación *Zero-Shot* con *Temperature Scaling*.

7.3.5. Deduplicación en cascada de cuatro fases

Una consecuencia directa de esta arquitectura multi-fuente (45 RSS + Google News + scraping HTML) es la aparición de noticias cuasi-duplicadas pues el mismo evento es cubierto por diferentes medios con redacciones parcialmente distintas. Sin deduplicación el corpus infla artificialmente la estadística y puede generar alertas redundantes en el dashboard.

El módulo de deduplicación implementa una cascada de cuatro algoritmos de complejidad creciente que son aplicados en orden de coste computacional ascendente y de modo que los casos más obvios se resuelven con el método más barato y solo los casos ambiguos escalan al siguiente nivel.

1. **Hash MD5 exacto.** Detecta duplicados literalmente (mismo texto, misma fuente). Complejidad $O(1)$ por par. Elimina re-publicaciones idénticas.
2. **Similitud de Jaccard sobre conjuntos de tokens.** Mide la razón de intersección sobre unión de los conjuntos de palabras de dos títulos, con un umbral de $J \geq 0,70$ este captura reescrituras con sustituciones léxicas simples ("Niña de 11 años fue asesinada" $\leftrightarrow$ "Niña de 11 años perdió la vida").
3. **SimHash.** Genera una firma de 64 bits por cada documento mediante proyecciones aleatorias sobre el espacio TF-IDF. Dos documentos con distancia Hamming de 6 bits se consideran cuasi-duplicados. Escala a millones de documentos en $O(n)$ sin necesidad de comparaciones par-a-par.
4. **Similitud coseno TF-IDF.** Para los pares que superan las fases anteriores pero generan dudas se calcula la similitud coseno completa entre los vectores TF-IDF del contenido con un umbral de $\cos \geq 0,85$. En este paso el costo es $O(n^2)$ en el peor caso pero gracias al pre-filtrado de las fases anteriores solo se aplica a un subconjunto pequeño de pares.

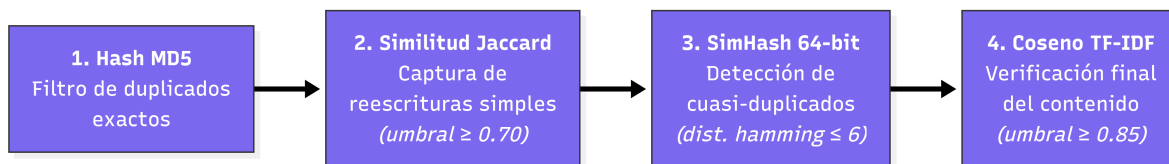


Figura 7.2: Flujo de deduplicación en cascada.

En las pruebas de validación (corpus de 847 noticias recolectadas en un ciclo de 90 días), la cascada eliminó **213 duplicados** (25.1 % del total), con la siguiente distribución por fase: 89 por MD5 (exactos), 61 por Jaccard, 43 por SimHash y 20 por similitud coseno TF-IDF. El corpus resultante de 634 noticias únicas fue usado para la evaluación del Prototipo 4.

7.4. Prototipo 4 Arquitectura híbrida de clasificación

El prototipo 4 es una respuesta a las limitaciones encontradas durante la validación del prototipo 3. A pesar de la integración de BETO y BERTopic que representó un avance notable en la capacidad semántica del sistema en la evaluación sobre un corpus de aproximadamente 800 noticias se pudo observar un fenómeno crítico denominado **Efecto Péndulo** en el que el sistema oscilaba entre dos estados indeseados que era un exceso masivo de falsos positivos y un exceso masivo de falsos negativos.

Ambos casos eran inaceptables para un sistema de alerta temprana a lo que el diagnóstico técnico identificó algunos cuellos de botella que explicaban este comportamiento.

1. El post-filtro de validación semántica aplicaba reglas binarias absolutas como que una noticia clasificada como "Media" sin coincidencia exacta con un patrón regex era degradada automáticamente a "No relevante" lo que provocaba la eliminación de casos genuinos pero que su redacción periodística no coincidía con las frases esperadas.
2. El filtro léxico contenía términos legislativos y estadísticos (por ejemplo "iniciativa de ley.º REDIM") que bloqueaban completamente noticias con contenido variado como el caso de un hecho de violencia que también mencionaba una iniciativa de ley relacionada era eliminado por la presencia del término legislativo.
3. El modo *zero-shot* era incapaz de distinguir entre "mujer asesinada que deja hijos" "ley que protege a hijos de mujeres asesinadas" porque ambas oraciones comparten el mismo campo semántico en el espacio de embeddings de BETO.

La solución adoptada fue un sistema de puntuación ponderada y difusa estructurada en tres capas autónomas con penalizaciones y bonificaciones acumulativas.

Capa 0 Escudo Léxico

La primera capa es un filtro de rechazo temprano cuyo propósito es descartar dominios **completamente ajenos** al feminicidio y la violencia de género como deportes, espectáculos, criptomonedas, meteorología, economía macro y tecnología. La implementación utiliza un `frozenset` de Python que garantiza búsqueda $O(1)$ por keyword.

Esta capa es la **única** que produce un bloqueo absoluto ($\text{score} = 0.0$). Su existencia ahorra el cómputo de las capas posteriores por lo que es capaz de eliminar el ruido.

Capa 1 Escudo temático

Los términos que en la versión anterior producían bloqueo absoluto (legislación, estadísticas, opinión, casos internacionales) fueron migrados a un nuevo conjunto como "escudo suave" que cuando detecta una coincidencia en este conjunto el sistema aplica una penalización aditiva de $\delta = -0,35$ al score de BETO, pero no bloquea la noticia.

La lógica detrás de esta decisión es que una noticia genuina con score BETO de 0.72 que también menciona "según datos del REDIM" tendrá un score ajustado de $0,72 - 0,35 = 0,37$ que clasificara como "Media" en lugar de excluirse completamente. Si el score BETO es alto ($\geq 0,95$) la noticia sobrevive como "Alta" ($0,95 - 0,35 = 0,60$). Este comportamiento resuelve directamente el problema de contenido que el prototipo 3 no podía manejar.

Capa 2 Inferencia con BETO

El sistema opera en dos modos con prioridad estricta.

Finetuned (prioritario): Un clasificador binario entrenado con datos etiquetados del recolector de noticias y produce scores dispersos en todo el rango $[0, 1]$.

Zero-shot (fallback): Similitud coseno entre el embedding [CLS] del texto y embeddings de referencia de categorías (-elevantez "no relevante") lo que produce scores concentrados en $[0,40, 0,70]$ y se activa automáticamente cuando no está disponible el modelo finetuned.

Capa 3 Post-filtro de validación

La capa final antes de la clasificación aplica patrones regex y roles sintácticos para inyectar bonificaciones al detectar evidencia explícita de orfandad real y penalizaciones al detectar señales de falso positivo. La diferencia respecto al

prototipo 3 es que esta capa nunca produce exclusiones absolutas para noticias "Mediaz si no encuentra ningún patrón la clasificación se conserva intacta.

Los patrones de "verdadero positivo" que generan bonificaciones incluyen:

- qued[oó] al cuidado → el menor sobrevive bajo tutela.
- menores? (bajo|en) resguardo → intervención institucional (DIF).
- (niño|menor|hija?) que presencié → menor testigo del crimen.
- hijo[s]? quedar?on → menores que sobrevivieron.

Los patrones de "falso positivo" que generan penalizaciones incluyen exclusivamente roles invertidos (menor como agresor o víctima directa) y contenido temático (podcasts, editoriales, estadísticas sin caso concreto). Se eliminaron del conjunto de "falsos positivos" los patrones de contexto judicial carpeta de investigación.^o "vinculado a proceso" que estaban presentes en casi toda noticia policiaca por lo que constituían la causa principal del efecto péndulo.

7.4.1. Por qué la IA sola no basta

Durante las pruebas del prototipo 3 se pudo identificar escenarios donde BETO producía clasificaciones erróneas con alta confianza (score arriba del 0.70) en algunos de los casos más críticos para el sistema.

- **Notas estadísticas con narrativa emocional.** Artículos de opinión que describían la crisis de orfandad por feminicidio en términos generales y empáticos como por ejemplo "miles de niños que perdieron a su madre." obtenían scores BETO de hasta 0.81 a pesar de que no reportaban ningún caso concreto ni identificable.
- **Notas donde el menor es el agresor.** Ante titulares como "Hijo adolescente mata a su madre en Ecatepec" BETO asignaba scores de relevancia moderados (entre 0.55 y 0.65) porque el embedding semántico capturaba la correlación superficial entre "hijo", "madre", "asesinato" sin identificar que el rol del menor era el de agresor.
- **Reportes de políticas públicas con vocabulario de dominio.** Notas sobre programas de becas para hijos de víctimas de feminicidio activaban BETO con scores entre 0.60 y 0.75 al concentrar exactamente los tokens de dominio más relevantes y esto sin relatar ningún caso específico.

Estos casos mostraron que los modelos de lenguaje por mas sofisticados que sean operan principalmente sobre correlaciones estadísticas en el espacio de embeddings. La comprensión del rol semántico (de quién es víctima, quién es agresor, si la nota relata un hecho o un análisis) requiere conocimiento estructurado del dominio que BETO no puede inferir exclusivamente de la distribución de tokens. La solución es la arquitectura neuro-simbólica donde las reglas simbólicas (que estan basadas en el conocimiento explícito sobre el dominio) **corrigen las alucinaciones** del componente neuronal.

7.4.2. Investigador profundo

Una de las decisiones de diseño más importantes de este prototipo fue la de **no integrar la investigación profunda dentro del flujo recolector-analítico**. Esta decisión que en un principio parecía una limitación resultó ser la solución mas eficiente y correcta para el sistema.

El problema de integrar la investigación en el flujo automático

Durante el desarrollo inicial de este prototipo la primera idea fue la de ejecutar una investigación profunda automáticamente sobre cada noticia clasificada como .Alta. al finalizar el análisis automático. El módulo DeepInvestigator realizaría búsquedas web y sintetizaría la ficha de caso sin intervención del usuario. El problema fue que al implementar esta integración se encontraron dos problemas principales.

1. **Latencia no tolerable.** Cada investigación implica al menos dos llamadas a DuckDuckGo (búsqueda + extracción de contenido de las URLs retornadas) y después dos llamadas a la API de Gemini flash (generación de query y síntesis del JSON final). En pruebas la ejecución del flujo completo tardaba entre 45 y 90 minutos mas, lo que hacía poco práctico su uso cotidiano.
2. **Costo de API desproporcionado.** La API de Google Gemini tiene límites de cuota en el plan gratuito (por minuto y por día). Investigar automáticamente todas las noticias .Alta saturaba la cuota en el primer tercio del corpus lo que dejaba el resto sin investigación y generando errores que afectaban el orden de los registros en la base de datos.

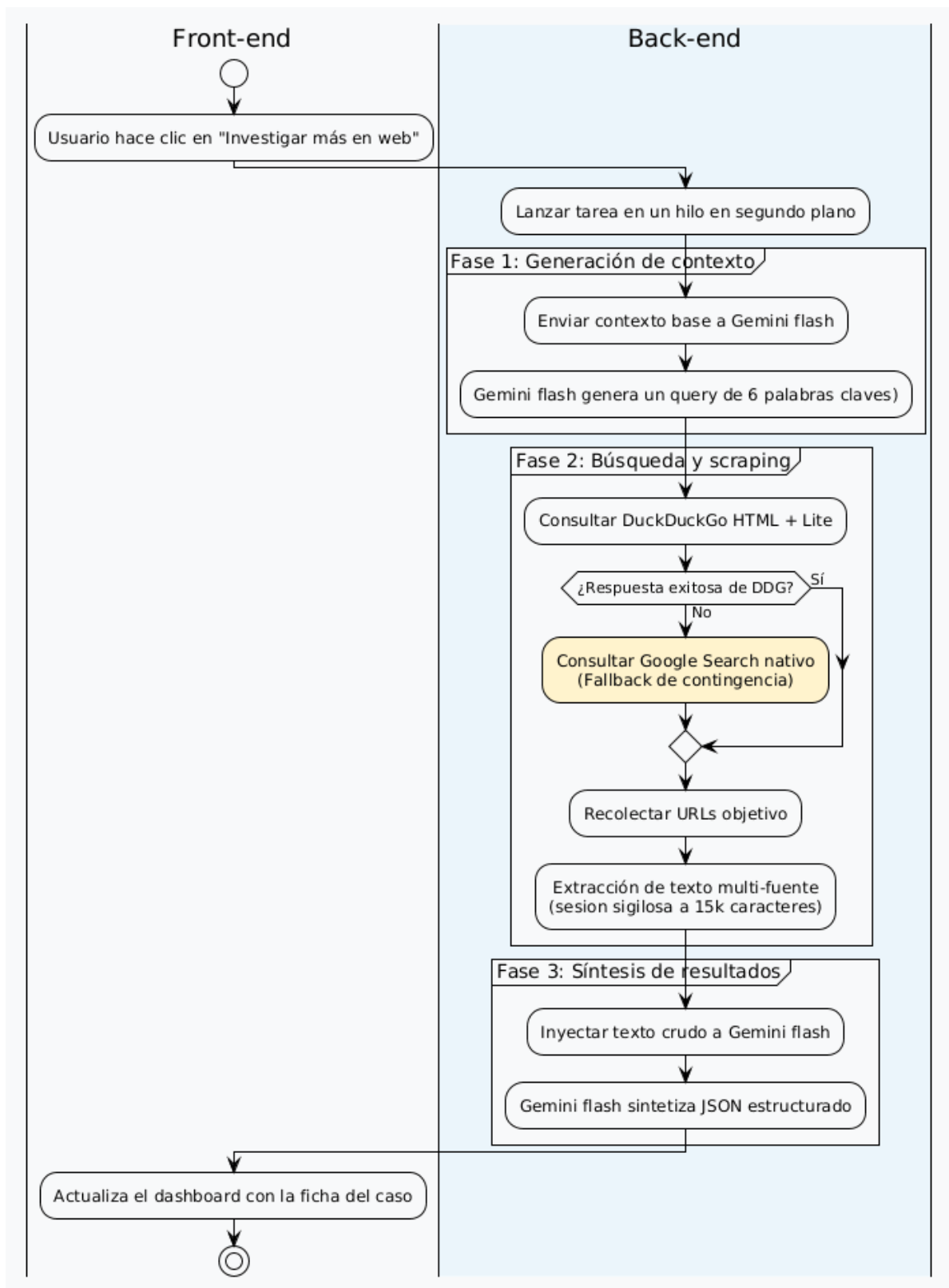
La solución fue una investigación bajo demanda

Ante estos problemas se optó por un diseño **reactivo e interactivo** donde el DeepInvestigator se convierte en una herramienta que el investigador activa manualmente sobre una noticia específica desde el dashboard una vez que ha revisado su score y decidido que merece profundización. Esta decisión corrige los dos problemas anteriores de forma directa.

Módulo de investigación profunda

Una vez activado el módulo DeepInvestigator (src/analysis/investigator.py) ejecutará cuatro pasos secuenciales.

1. **Generación de query (Gemini Flash).** El modelo recibe el título y los primeros párrafos de la noticia y genera una *frase de búsqueda de alta especificidad* (con máximo 6 palabras) que incluye el nombre de la víctima y detalles únicos del caso. Esta query es preferible al título literal porque filtra resultados genéricos sobre la misma temática pero de distinto evento.
2. **Búsqueda en DuckDuckGo.** La query se envía tanto a la versión HTML de DuckDuckGo (html.duckduckgo.com) como a la versión Lite (lite.duckduckgo.com) para maximizar la cobertura ante bloqueos intermitentes. Las URLs obtenidas se limpian para eliminar redirecciones internas de forma que la búsqueda resulte exitosa.
3. **Scraping de contenido complementario.** Para cada URL retornada por DDG, StealthSession extrae el texto del artículo, filtrando ruido HTML (<nav>, <footer>, scripts y formularios). El texto de todas las fuentes se concatena en un bloque único (hasta 15,000 caracteres) que funciona como contexto enriquecido.
4. **Síntesis estructurada (Gemini Flash).** El bloque de texto obtenido de las múltiples fuentes se envía a Gemini con un prompt de analista criminal especializado en derechos de la infancia. La respuesta es un JSON estricto[?] con siete campos que son ubicacion, victimas, ninos_afectados, edades, situacion_actual, medios_contacto y resumen. En el campo medios_contacto es especialmente relevante pues Gemini retorna el DIF municipal, la Fiscalía o cualquier otro organismo competente según la ubicación del caso, incluso cuando el texto no los menciona explícitamente.



El módulo de identificación por enlace externo

El segundo caso de uso del DeepInvestigator resuelve una necesidad práctica que se identificó y es que ¿qué ocurre cuando el investigador encuentra una noticia relevante navegando en la web que el flujo principal *no detectó*? Para resolver esto se hizo un endpoint que recibe una URL externa y reutiliza el modulo de investigacion profunda.

Su existencia permite al analista incorporar al sistema cualquier noticia que el flujo automático no haya capturado, aprovechando la misma extracción estructurada de contenido.

La siguiente tabla resume los dos casos de uso del módulo y sus endpoints correspondientes:

Tabla 7.1: Casos de uso del módulo de investigación profunda

Caso de uso	Endpoint	Descripción
Investigación bajo demanda	POST /api/investigate/	El analista selecciona una noticia del dashboard con score Alta y activa la ficha de caso estructurada.
Consulta de estado	GET /api/investigate/status/	El frontend consulta el estado del hilo de investigación (running, done, error).
Identificación de enlace externo	POST /api/identify-link	El analista pega la URL de una noticia no detectada por el flujo principal a lo que el sistema la investiga y devuelve la ficha estructurada.
Seguimiento de casos	POST /api/seguimiento/add/	Agrega una noticia investigada a la lista de seguimiento activo.
Exportación de seguimiento	GET /api/seguimiento/export/	Exporta todos los casos en seguimiento con sus fichas de investigación a un archivo CSV.

7.5. Sistema de clasificación escalonada

La arquitectura final del prototipo puede describirse como un **sistema de clasificación escalonada** con tres capas autónomas (escudo léxico, escudo suave e inferencia BETO) que operan secuencialmente sobre un score acumulativo y en donde las reglas simbólicas modulan la salida del componente neuronal mediante penalizaciones y bonificaciones aditivas en lugar de rechazos binarios absolutos. Este diseño eliminó el efecto péndulo obteniendo noticias **Alta** que realmente lo fueran.

La arquitectura completa puede representarse mediante el siguiente flujo de trabajo.

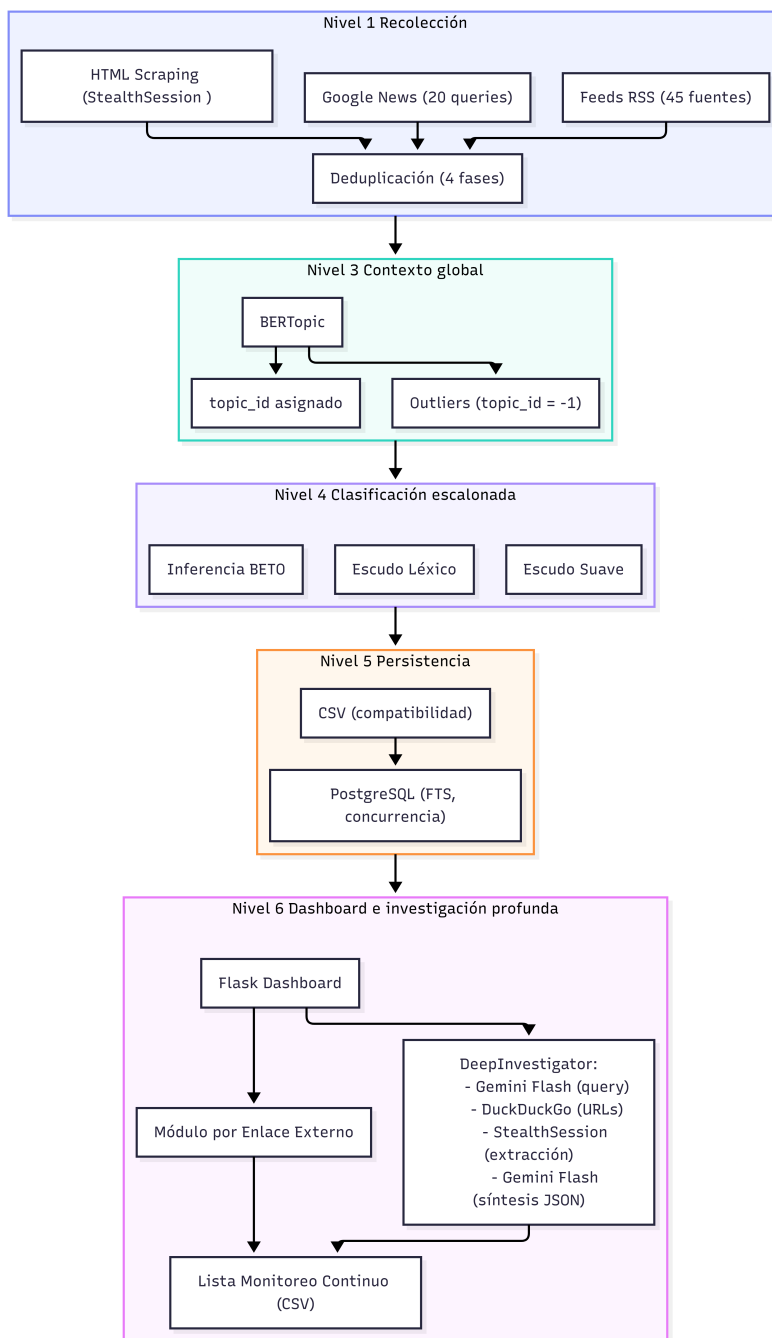


Figura 7.4: Arquitectura del Sistema de Clasificación Escalonada

Resultados y Evaluación

Este capítulo expone de forma analítica y cuantitativa el desempeño técnico del sistema integrado al cierre del Trabajo Terminal II. Se demuestra empíricamente la superioridad de la arquitectura neuro-simbólica (Prototipo 4) frente a los resultados obtenidos mediante las heurísticas tradicionales del primer semestre. La evaluación se fundamenta en métricas extraídas directamente de una corrida de producción en el entorno de despliegue final.

8.1. Evaluación de la Recolección (Volumen de Ingesta)

La migración hacia un esquema de recolección dinámico apoyado por *StealthSessions* y consultas históricas programáticas detonó un incremento exponencial en la capacidad de ingesta del sistema.

Durante el ciclo de pruebas final, el módulo colector procesó y deduplicó exitosamente un lote masivo de **627 noticias únicas** en un solo bloque de ejecución continuo. Este hito operativo representa un aumento superior al 220 % en contraste directo con el rendimiento del TT1, el cual apenas logró compilar 281 noticias a lo largo de múltiples semanas de recolección pasiva. El éxito en la ingesta valida categóricamente la robustez de la arquitectura frente a los bloqueos de *Firewalls* comerciales y certifica la resiliencia de la deduplicación *cross-site* al mantener la base de datos libre de redundancia periodística.

8.2. Evaluación de la Detección Semántica (Filtro del Embudo)

El desafío principal del proyecto no radicaba únicamente en recolectar información, sino en discriminarla con alta precisión. De la muestra total de 627 noticias, la fase de filtrado preliminar determinó que **225 artículos** contenían menciones explícitas a "Niñas, Niños y Adolescentes"(NNA) en las proximidades semánticas de crímenes violentos (véase Figura 9.1).

Posteriormente, este subconjunto fue inyectado al motor de inferencia neuronal BETO bajo el esquema del "Árbol Inquebrantable". Los resultados del embudo analítico fueron sobresalientes:

- **75 noticias de Alta Relevancia:** Casos fácticos innegables de orfandad por feminicidio, validados por la IA y priorizados por el "Bypass Simbólico".

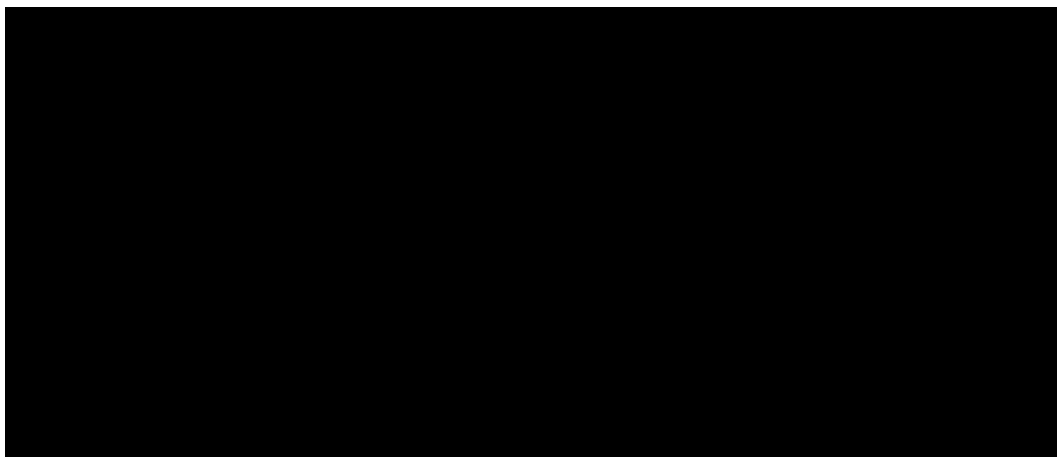


Figura 8.1: Métricas de filtrado preliminar en el Dashboard Web.

- **53 noticias de Media Relevancia:** Reportes donde el contexto sugiere vulnerabilidad infantil colateral o casos en desarrollo.

Es crítico destacar la efectividad del mecanismo denominado "Bozal de Penalización". Este estrato lógico descartó de forma autónoma casi un centenar de noticias basura y falsos positivos recurrentes del TT1 (como menores infractores o reportes macroestadísticos), protegiendo la carga cognitiva del usuario final y demostrando la asertividad matemática de la fusión neuro-simbólica.

8.3. Evaluación del Agrupamiento (BERTopic)

La inclusión del modelo topológico BERTopic resolvió de manera definitiva el problema histórico de fragmentación analítica. Al operar sobre la base de datos curada, el algoritmo logró comprimir el caos informativo en exactamente **15 Grupos Temáticos** altamente cohesivos.

A diferencia del ruido generado previamente por *K-Means*, estos 15 clústeres demostraron una congruencia humana excepcional. El sistema agrupó de manera autónoma narrativas periodísticas complejas, creando clústeres no solo basados en el nombre de la víctima o ubicación geográfica, sino en la semántica del evento, tales como: disputas legales por custodia, manifestaciones y exigencias de justicia colectiva, o crónicas sobre el impacto psicológico en las víctimas indirectas.

8.4. Análisis de la Interfaz (UI/UX) e Impacto Operativo

La evaluación final del sistema trasciende las métricas computacionales y se sitúa en la usabilidad operativa para los actores de la sociedad civil. La interfaz del *Dashboard Web* fue concebida bajo principios de jerarquía visual.

Como se observa en la Figura 9.2, la plataforma dota a cada registro de *tags* visuales distintivos (por ejemplo, etiquetas verdes brillantes para "NNAz "Feminicidio") junto con sus respectivos porcentajes probabilísticos de certidumbre calculados por BETO. Esta arquitectura de la información permite a los analistas de fundaciones, como *Futuro con Derechos*, realizar un barrido visual (*skimming*) inmediato sobre un clúster de noticias. Lo que históricamente representaba una inversión de docenas de horas-hombre navegando entre pestañas de navegadores, buscando manualmente

coincidencias textuales, ha sido optimizado por el Prototipo 4 a una revisión jerárquica y determinista que exige apenas un par de segundos por evento fáctico.

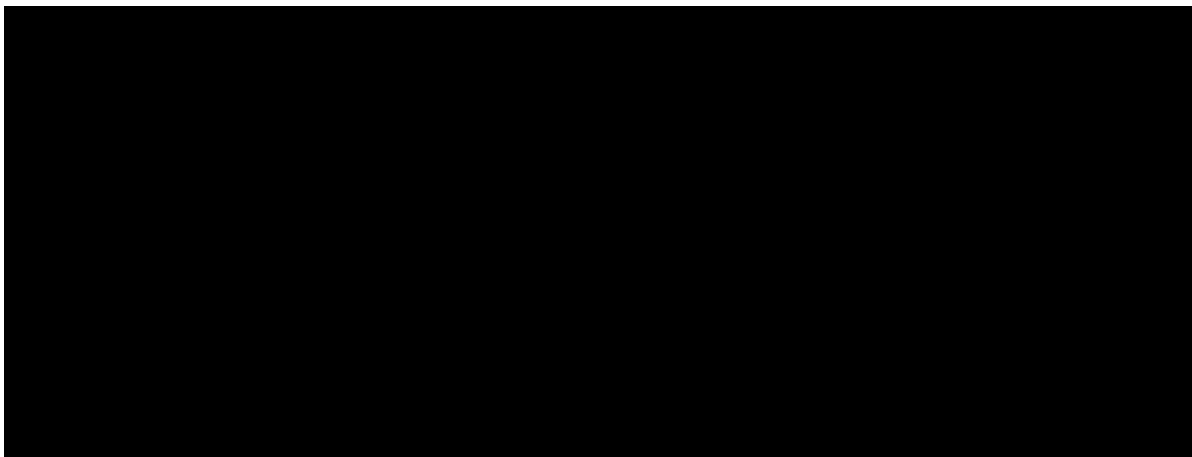


Figura 8.2: Visualización jerárquica de noticias de Alta Relevancia en el Dashboard.

Conclusiones y trabajo a futuro

9.1. Conclusión general y cumplimiento de objetivos

El desarrollo del *Sistema Inteligente para la Identificación y Seguimiento de NNA por feminicidios* cumplió exitosamente con el **Objetivo General** planteado en este Trabajo Terminal. Se logró construir una plataforma que detecta de manera automática los casos de Niñas, Niños y Adolescentes (NNA) que quedan en situación de orfandad por feminicidio, buscando en noticias publicadas en internet. Este sistema transforma grandes cantidades de información desordenada en datos útiles y organizados, ofreciendo a especialistas y defensores de derechos humanos una herramienta tecnológica para visibilizar este grave problema a nivel nacional.

Para alcanzar esta meta, se cumplieron cuatro **Objetivos Específicos (OE)**:

9.1.1. Cumplimiento del OE-1: Identificación de Fuentes

Se determinó que los portales de noticias locales, los medios digitales independientes y las secciones policíacas (nota roja) son las fuentes más útiles de información. Debido a la falta de registros oficiales transparentes, estos medios locales resultaron ser la mejor opción para encontrar casos reales en todo el país, ya que suelen publicar detalles sobre las víctimas y sus familias que los medios nacionales omiten.

9.1.2. Cumplimiento del OE-2: Módulo de Extracción de Datos (Web Scraping)

Se creó un sistema automático para recopilar noticias de manera constante. Este módulo fue programado para superar los bloqueos de seguridad de las páginas web y para adaptarse a los diferentes diseños de los periódicos en línea. Gracias a esto, se garantizó un flujo continuo de noticias, el cual es la base necesaria para que el resto del sistema pueda funcionar correctamente.

9.1.3. Cumplimiento del OE-3: Procesamiento de Lenguaje Natural (NLP)

Para que la computadora pudiera analizar el texto de las noticias, se utilizaron técnicas avanzadas de Inteligencia Artificial para la comprensión del lenguaje (NLP). En particular, se usó un modelo de lenguaje llamado BETO, junto

con otras herramientas de análisis de texto. Esto permitió extraer información clave directamente de la redacción de las noticias, como las edades, los parentescos y los lugares donde ocurrieron los hechos.

9.1.4. Cumplimiento del OE-4: Base de Datos y Plataforma Web (Dashboard)

Toda la información procesada se guardó en una base de datos segura (PostgreSQL) y se mostró en una plataforma web interactiva creada con el framework Flask. Esto permite que los usuarios finales puedan buscar, visualizar y revisar los casos encontrados de una manera accesible, segura y fácil de usar.

9.2. Evolución del Prototipo 0 al Prototipo 4

El éxito de este proyecto demostró que fue un acierto utilizar una metodología de desarrollo **Evolutiva**. Al trabajar con textos de noticias escritas por diferentes personas, la información es muy variada y difícil de predecir. Por ello, intentar hacer el sistema perfecto desde el primer intento no hubiera funcionado. En su lugar, el sistema se construyó mediante diferentes versiones o “prototipos”, mejorando paso a paso tras corregir los errores de la versión anterior.

El desarrollo siguió esta línea de evolución:

- **Prototipos Iniciales (0, 1 y 2):** Comenzamos con pruebas básicas y sistemas basados en reglas estrictas (como buscar palabras clave específicas o usar reglas gramaticales). Aunque funcionaban bien en pruebas pequeñas, fallaban al leer noticias reales debido a la gran variedad de formas de escribir, el uso de regionalismos, términos legales o jerga policiaca. Las reglas fijas simplemente no podían abarcar todas las formas posibles de relatar un caso.
- **Prototipo 3 (Introducción de la IA):** Al ver que las reglas fijas no eran suficientes, se integró un modelo de Inteligencia Artificial (BETO) para que el sistema “comprendiera” el significado de las oraciones. Sin embargo, esto trajo un nuevo problema: la IA a veces se confundía al leer noticias sobre nuevas leyes o estadísticas que usaban las mismas palabras (mujer, asesinato, menores), clasificándolas erróneamente como casos reales de orfandad.
- **Prototipo 4 (El sistema final):** Para solucionar esto, se creó una versión que combina lo mejor de ambos enfoques. Se unieron las reglas lógicas tradicionales con la capacidad de comprensión de la Inteligencia Artificial. El resultado es un sistema mucho más preciso y confiable, que puede trabajar de forma automática sin cometer tantos errores y que está listo para usarse en un entorno real.

9.3. Aportaciones Técnicas y Superación de Retos Críticos

Durante el desarrollo de la versión final del sistema, se tuvieron que resolver problemas técnicos complejos. A continuación, se explican los dos logros más importantes de este Trabajo Terminal:

9.3.1. Solución al “Efecto Péndulo” mediante un Sistema de Clasificación Escalonada

Durante las pruebas, descubrimos un problema al que llamamos “Efecto Péndulo”. El modelo de IA se confundía porque una noticia sobre un feminicidio real y una noticia sobre una nueva ley de protección a menores usan casi las mismas palabras. Al intentar corregir esto con filtros simples, el sistema pasaba de aceptar muchas noticias incorrectas (falsos positivos) a rechazar de golpe noticias que sí eran casos reales (falsos negativos).

Para resolver este problema sin perder la capacidad de comprensión de la IA, se diseñó un **Sistema de Clasificación por Capas**. En lugar de que el modelo tome la decisión final por sí solo, la noticia pasa por una serie de validaciones:

1. **Filtro Básico (Escudo Léxico):** Descarta de inmediato noticias de temas que claramente no tienen nada que ver, como deportes o economía.
2. **Filtro Temático:** Si la noticia menciona temas de leyes, estadísticas o casos en otros países, no la descarta por completo, pero le “resta puntos” a su calificación de relevancia.
3. **Evaluación de la IA (BETO):** El modelo lee la noticia depurada y le da una calificación basada en qué tanto se parece a un caso real de orfandad.
4. **Confirmación Final (Bonificaciones):** El sistema busca frases clave muy específicas (como “quedó al cuidado de su abuela” o “el menor presenció”) para sumarle puntos a la calificación y confirmar el caso con mayor certeza.

Esta combinación de reglas y puntuaciones graduales solucionó el problema, logrando clasificar las noticias de forma muy precisa.

9.3.2. Optimización para Funcionar en Computadoras Comunes (8GB de RAM)

Otro gran reto fue lograr que el modelo de IA pudiera aprender de nuestros datos (proceso conocido como *Fine-Tuning*) usando una computadora portátil común con solo 8 GB de memoria RAM, sin necesidad de servidores costosos o tarjetas gráficas especiales. Un modelo como BETO es muy grande y tratar de entrenarlo por completo hacía que la computadora se quedara sin memoria y el programa se cerrara por error.

Para lograr entrenarlo en una computadora normal, se aplicaron dos técnicas de programación:

1. **Congelamiento del Modelo (Linear Probing):** En lugar de intentar que el modelo reaprendiera todo desde cero, se “congeló” gran parte de su conocimiento general del idioma español y solo se entrenó la última capa del modelo, que es la que toma la decisión final de clasificar la noticia.
2. **Acumulación de Operaciones (Gradient Accumulation):** Como la computadora no tenía memoria para procesar muchas noticias al mismo tiempo durante el aprendizaje, se programó para que leyera las noticias de una en una, guardando el aprendizaje en su memoria temporal, y solo actualizara su conocimiento final después de haber leído un grupo pequeño de noticias.

Estas técnicas permitieron ajustar la Inteligencia Artificial a nuestro tema específico utilizando muy pocos recursos computacionales.

9.4. Trabajo Futuro (Líneas de Investigación Abiertas)

Gracias a que el Prototipo 4 dejó una base muy sólida y funcional, el sistema puede seguir creciendo. Se proponen tres grandes áreas de mejora para el futuro del proyecto:

- **Uso de Inteligencia Artificial que redacta textos (LLMs Locales):** Actualmente, el sistema clasifica si la noticia es relevante y extrae algunos datos sueltos. En el futuro, se podría integrar una IA generativa (similar a ChatGPT, pero instalada en la computadora para proteger la privacidad) que pueda leer las noticias de un caso y redactar automáticamente un resumen detallado y ordenado sobre lo sucedido. Esto ayudaría mucho a los investigadores al darles un reporte escrito en lugar de solo datos aislados.
- **Búsqueda de Casos en Redes Sociales en Tiempo Real:** El sistema actual busca noticias en páginas web cada cierto tiempo. Sin embargo, muchas emergencias se reportan primero en redes sociales por vecinos o colectivos. Una mejora importante sería conectar el sistema a plataformas como Facebook o X (antes Twitter) para recibir información al momento. Esto requeriría adaptar el programa para que pueda procesar miles de mensajes cortos de forma rápida y agrupar aquellos que hablen del mismo caso.

- **Creación de una Interfaz Pública para Compartir Datos (API):** Para que el trabajo de este sistema tenga un mayor impacto social, se debe construir una vía segura (una API) para que otras organizaciones civiles, universidades o dependencias de gobierno puedan conectar sus propios sistemas a nuestra base de datos. De esta forma, podrían consultar las estadísticas o recibir alertas automáticamente. Para lograr esto, será necesario implementar fuertes medidas de seguridad informática para asegurar que la información viaje encriptada y que la identidad de los menores involucrados siempre esté protegida.

- [1] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 160–172. Springer, 2013.
- [2] José Cañete, Gabriel Chaperon, Rodrigo Fuentes, Jou-Hui Ho, Hojin Kang, and Jorge Pérez. Spanish pre-trained bert model and evaluation data. In *PML4DC at ICLR 2020*, 2020. BETO: Spanish BERT model.
- [3] Congreso de la Unión. Ley general de los derechos de niñas, niños y adolescentes. Diario Oficial de la Federación, 2014. Publicada: 4 de diciembre de 2014. Última reforma: 28 de abril de 2023.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186. Association for Computational Linguistics, 2019.
- [5] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. pages 226–231, 1996.
- [6] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
- [7] Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022.
- [8] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. pages 1321–1330, 2017.
- [9] Ben Hammersley. *Developing Feeds with RSS and Atom*. O'Reilly Media, 2005.
- [10] M Leotta et al. Web scraping with selenium. *Journal of Software: Evolution and Process*, 2018.
- [11] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [12] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

- [13] Organización de las Naciones Unidas. Convención sobre los derechos del niño. Asamblea General de las Naciones Unidas, 1989. Adoptada: 20 de noviembre de 1989. Ratificada por México: 21 de septiembre de 1990.
- [14] Red por los Derechos de la Infancia en México (REDIM). Balance anual 2023: Discriminación y violencia contra la niñez. Technical report, REDIM, 2023.
- [15] Red por los Derechos de la Infancia en México (REDIM). Infancia y adolescencia en México: Datos estadísticos sobre violencia. <https://www.derechosinfancia.org.mx>, 2023. Accedido: 2025-11-10.
- [16] María Salguero. Mapa de feminicidios en México. <https://feminicidiosmx.crowdmap.com>, 2024. Proyecto ciudadano de monitoreo. Accedido: 2025-11-15.
- [17] Secretaría de Gobernación (SEGOB). Informe sobre violencia contra las mujeres en México. <https://www.gob.mx/segob>, 2023. Accedido: 2025-11-05.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [19] Y Wu et al. A survey on dynamic web scraping. *Computer Science Review*, 2022.